Variables & inferred typing

This lesson discusses the basics of variable declaration, initialization and inferred typing.

WE'LL COVER THE FOLLOWING

- Variables Declaration
- Variable Initialization

Go is often referred to as a “simple” programming language, a language that can be learned in a few hours if you are familiar with any basic programming language. Go was designed to feel familiar and to stay as simple as possible, the entire language specification fits in just a few pages.

There are a few concepts we are going to explore before writing our first application.

Variables Declaration

The `var` statement declares a list of variables. The name of the variable comes first, and the type of variable is declared after it.

Environment Variables

Key:	Value:
GOPATH	/go

```
var (
        name     string
        age      int
        location string
)
```

Or even

Environment Variables

Key:	Value:
GOPATH	/go

```
var (
        name, location  string
        age             int
)
```

Variables can also be declared one by one:

Environment Variables

Key:	Value:
GOPATH	/go

```
var name     string
var age      int
var location string
```

Variable Initialization

A `var` declaration can include initializers, one per variable.

Environment Variables

Key:	Value:
GOPATH	/go

```
var (
        name     string = "Prince Oberyn"
        age      int    =  32
        location string = "Dorne"
)
```

If an initializer is present, the type can be omitted, the variable will take the type of the initializer (inferred typing).

Environment Variables

Key:	Value:
GOPATH	/go

```
var (
        name     = "Prince Oberyn"
```

```
        age      =  32
        location = "Dorne"
)
```

You can also initialize multiple variables at once.

Environment Variables

Key:	Value:
GOPATH	/go

```
var (
        name, location, age = "Prince Oberyn", "Dorne", 32
)
```

Inside a function, the `:=` short assignment statement can be used in place of a `var` declaration with implicit type.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func main() {
        name, location := "Prince Oberyn", "Dorne"
        age := 32
        fmt.Printf("%s age %d from %s ", name, age, location)
}
```

A variable can contain any type, including functions:

Environment Variables

Key:	Value:
GOPATH	/go

```
func main() {
        action := func() { //action is a variable that contains a function
                //doing something
        }
        action()
}
```

Outside a function, every construct begins with a keyword (`var`, `func` and so on) and the `:=` construct is not available.

Use Go's declaration Syntax to read more about the Go syntax.

Let's take a look at how *constants* are declared in the next chapter.

Constants

This lesson explains how to declare const type variables.

WE'LL COVER THE FOLLOWING

- Declaration
- Example

Declaration

Constants are declared like variables, but with the `const` keyword.

Constants can only be a **character**, **string**, **boolean**, or **numeric values** and cannot be declared using the `:=` syntax. An *untyped* constant takes the type needed by its context.

Environment Variables

Key:	Value:
GOPATH	/go

```
const Pi = 3.14
const (
        StatusOK                   = 200
        StatusCreated              = 201
        StatusAccepted             = 202
        StatusNonAuthoritativeInfo = 203
        StatusNoContent            = 204
        StatusResetContent         = 205
        StatusPartialContent       = 206
)
```

Let's take a look at an example below demonstrating this concept.

Example

Environment Variables

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

const (
        Pi    = 3.14
        Truth = false
        Big   = 1 << 62
        Small = Big >> 61
)

func main() {
        const Greeting = "ハローワールド" //declaring a constant
        fmt.Println(Greeting)
        fmt.Println(Pi)
        fmt.Println(Truth)
        fmt.Println(Big)
}
```

> **Note:** The *left-shift operator* `(<<)` shifts its first operand left by the number of **bits** specified by its *second operand*. The type of the *second operand* **must** be an `int` or a type that has a predefined implicit numeric conversion to `int`. The *right-shift operator* `(>>)` shifts its *first operand* right by the number of **bits** specified by its *second operand*.

Now that you know how to declare constants from the above examples. Let's move on and read about printing constants/variables.

Printing

This lesson explains how to print variables and constants.

WE'LL COVER THE FOLLOWING

- Printing Constants and Variables

Printing Constants and Variables

While you can print the value of a variable or constant using the built-in `print` and `println` functions, the more idiomatic and flexible way is to use the `fmt` package

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func main() {
        cylonModel := 6
        fmt.Println(cylonModel)
}
```

`fmt.Println` prints the passed in variables' values and appends a newline. `fmt.Printf` is used when you want to print one or multiple values using a defined format specifier (`%d`, `%s`).

Environment Variables

Key:	Value:

GOPATH	/go

```
package main
import "fmt"

func main() {
        name := "Caprica-Six"
        aka := fmt.Sprintf("Number %d", 6)
        fmt.Printf("%s is also known as %s",
                name, aka) //printing variables within the string
}
```

Read the `fmt` package documentation to see the available flags to create a format specifier.

The next lesson will touch upon packages and imports. Read on to find out more.

Packages and Imports

This lesson explains how to import different libraries and packages in GO

WE'LL COVER THE FOLLOWING

- Packages
- Import statement examples

Packages

Every Go program is made up of packages. Programs start running in package `main`.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func main() {
        fmt.Printf("Hello, World!\n")
}
```

If you are writing an executable code (versus a library), then you need to define a `main` package and a `main()` function which will be the entry point to your software.

By convention, the package name is the same as the last element of the import path. For instance, the “math/rand” package comprises files that begin with the statement package rand.

Import statement examples

Environment Variables

Key:	Value:
GOPATH	/go

```
import "fmt"
import "math/rand"
```

Or grouped:

Environment Variables

Key:	Value:
GOPATH	/go

```
import (
    "fmt"
    "math/rand"
)
```

Check out the two links below for more information about *packages* and *imports* in GO.

- Go Tour: Packages
- Go Tour: Imports

Below are two examples showing how to use each of these imports.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "math/rand"
)

func main() {
        fmt.Println("My favorite number is", rand.Intn(10))
}
```

Down below is another example to help further understand how imported packages can be used.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "math"
)

func main() {
        fmt.Printf("Now you have %g problems.", math.Sqrt(7))
}
```

Usually, nonstandard lib packages are namespaced using a *web url*. For instance, there is some Rails logic ported to GO, including the cryptography code used in *Rails 4*. The source code is hosted on **GitHub** containing a few packages, in the following repository: http://github.com/mattetti/goRailsYourself

To import the crypto package, you would need to use the following import statement:

Environment Variables

Key:	Value:
GOPATH	/go

```
import "github.com/mattetti/goRailsYourself/crypto"
```

The next lesson further explains this in more detail. Read on to find out more!

Code Location

This lesson explains in detail how to import a code from a location using Go get command

WE'LL COVER THE FOLLOWING

- Getting code location

Getting code location

The path `github.com/mattetti/goRailsYourself/crypto` basically tells the compiler to import the *crypto package* available. It doesn't mean that the compiler will automatically pull down the repository, so where does it find the code?

You need to pull down the code yourself. The easiest way is to use the `go get` command provided by Go.

Environment Variables

Key:	Value:
GOPATH	/go

```
$ go get github.com/mattetti/goRailsYourself/crypto
```

This command will pull down the code and put it in your *Go path*. When installing Go, we set the `GOPATH` environment variable and that is what's used to store binaries and libraries. That's also where you should store your code (your workspace).

Environment Variables

Key:	Value:
GOPATH	/go

```
$ ls $GOPATH
bin     pkg     src
```

The `bin` folder will contain the Go compiled binaries. You should probably add the bin path to your system path.

The `pkg` folder contains the compiled versions of the available libraries so the compiler can link against them without recompiling them.

Finally the `src` folder contains all the Go source code organized by import path:

Environment Variables

Key:	Value:
GOPATH	/go

```
$ ls $GOPATH/src
bitbucket.org   code.google.com github.com      launchpad.net
```

Environment Variables

Key:	Value:
GOPATH	/go

```
$ ls $GOPATH/src/github.com/mattetti
goblin                  goRailsYourself         jet
```

When starting a new program or library, it is recommended to do so inside the `src` folder, using a fully qualified path (for instance: `github.com/<your username>/<project name>`)

Now that you're done reading about *import* let's learn about *export* in the next chapter.

Exported names

This lesson discusses how to refer to exported names and how to use them after importing their package

WE'LL COVER THE FOLLOWING

- How to Export

How to Export

After importing a package, you can refer to the names it exports (meaning variables, methods, and functions that are available from outside of the package). In Go, a name is exported if it begins with a capital letter. `Foo` is an exported name, as is `FOO`. The name `foo` is not exported.

See the difference between:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
    "fmt"
    "math"
)

func main() {
    fmt.Println(math.pi) //lowercase name
}
```

As you can see the code above gave an error as lowercase cannot be exported. On the other hand, the code below will run fine as `Pi` is begins with a capital letter so can be exported.

letter so can be exported.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
    "fmt"
    "math"
)

func main() {
    fmt.Println(math.Pi) //uppercase name
}
```

`Pi` is exported and can be accessed from outside the package, while `pi` isn't available.

Use the provided Go documentation or godoc.org to find exported names.

Below is another *exported names* example:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    fmt.Printf("HTTP status OK uses code: %d", http.StatusOK)
}
```

The next lesson will cover *functions* and *return* values. Read on to find out more!

Functions and Return values

This lesson discusses functions, return values, named results and signature

WE'LL COVER THE FOLLOWING

- Functions
- Return values
  - Named Results

Functions

When declaring functions, the type comes after the variable *name* in the inputs.

The return type(s) are then specified after the function name and inputs, before writing the definition. Functions can be defined to return any number of values and each of them are put here.

Given below is an example function definition:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func add(x int, y int) int {
    return x + y
}

func main() {
    fmt.Println(add(42, 13))
}
```

In the following example, instead of declaring the type of each parameter, we only declare one type that applies to both.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func add(x, y int) int {
    return x + y
}

func main() {
    fmt.Println(add(42, 13))
}
```

Return values

In the following example, the `location` function returns two string values.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func location(city string) (string, string) {
        var region string
        var continent string

        switch city {
        case "Los Angeles", "LA", "Santa Monica":
                region, continent = "California", "North America"
        case "New York", "NYC":
                region, continent = "New York", "North America"
        default:
                region, continent = "Unknown", "Unknown"
        }
        return region, continent
}
```

```
}

func main() {
        region, continent := location("Santa Monica")
        fmt.Printf("Matt lives in %s, %s", region, continent)
}
```

Named Results

Functions take parameters. In Go, functions can return multiple “result parameters”, not just a single value. They can be named and act just like variables.

If the result parameters are named, a return statement without arguments returns the current values of the results.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func location(city string) (region, continent string) {
        switch city {
        case "Los Angeles", "LA", "Santa Monica":
                region, continent = "California", "North America"
        case "New York", "NYC":
                region, continent = "New York", "North America"
        default:
                region, continent = "Unknown", "Unknown"
        }
        return //returning region and continent
}

func main() {
        region, continent := location("Santa Monica")
        fmt.Printf("Matt lives in %s, %s", region, continent)
}
```

I personally recommend against using *named* return parameters because they often cause more confusion than they save time or help clarify your code.

In the next lesson, we will discuss *pointers*. Read on to find out more!

Pointers

This lesson discusses pointers and how to pass them as arguments in GO

WE'LL COVER THE FOLLOWING

- Go and Pointers

Go and Pointers

Go has *pointers*, but no pointer arithmetic. *Struct* fields can be accessed through a struct pointer. The indirection through the pointer is **transparent** (you can directly call fields and methods on a pointer).

Note that by *default* Go passes arguments by value (copying the arguments), if you want to pass the arguments by reference, you need to pass pointers (or use a structure using reference values like slices and maps.

To get the pointer of a value, use the `&` symbol in front of the value; to dereference a pointer, use the `*` symbol.

Methods are often defined on pointers and not values (although they can be defined on both), so you will often store a pointer in a variable as in the example below:

Environment Variables

Key:	Value:
GOPATH	/go

```
client := &http.Client{}
resp, err := client.Get("http://gobootcamp.com")
```

Now that you're clear about *pointers* in GO. The next lesson will discuss the concept of *mutability*.

Mutability

This lesson discusses mutability and how to use it with help of pointers in GO

In Go, only *constants* are **immutable**. However, because arguments are passed by value, a function receiving a value argument and mutating it, won't mutate the original value.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

type Artist struct {
        Name, Genre string
        Songs       int
}

func newRelease(a Artist) int { //passing an Artist by value
        a.Songs++
        return a.Songs
}

func main() {
        me := Artist{Name: "Matt", Genre: "Electro", Songs: 42}
        fmt.Printf("%s released their %dth song\n", me.Name, newRelease(me))
        fmt.Printf("%s has a total of %d songs", me.Name, me.Songs)
}
```

As you can see the total amount of songs on the `me` variable's value wasn't changed. To mutate the passed value, we need to pass it by reference, using a pointer.

Environment Variables

Key: Value:

GOPATH /go

```
package main
import "fmt"

type Artist struct {
        Name, Genre string
        Songs       int
}

func newRelease(a *Artist) int { //passing an Artist by reference
        a.Songs++
        return a.Songs
}

func main() {
        me := &Artist{Name: "Matt", Genre: "Electro", Songs: 42}
        fmt.Printf("%s released their %dth song\n", me.Name, newRelease(me))
        fmt.Printf("%s has a total of %d songs", me.Name, me.Songs)
}
```

The only change between the two versions is that `newRelease` takes a pointer to an `Artist` value and when I initialize our `me` variable, I used the `&` symbol to get a pointer to the value.

Another place where you need to be careful is when calling methods on values as explained a bit later.

This marks the end of this chapter. In the next chapter, we will discuss basic types and conversions.

Quiz on Go Variables

A quick quiz to test understanding of this chapter!

1

Which of the following is NOT a correct way to declare a variable in Go?

2

In Go, functions can return just a single value, the “result parameter”.

3

Which of the following is Not true about Constants?

4

In Go, a name is exported if it begins with a capital letter?

5

In Go, only constants are mutable

Check Answers

Basic Types

This lesson list of inbuilt variable types in Go.

WE'LL COVER THE FOLLOWING

- List of Built In Types
  - Common Types
  - Numeric types
- Example

List of Built In Types

The following are the built in types in Go:

Common Types

`bool`	true or false
`string`	an array of characters

Numeric types

`uint`	either 32 or 64 bits.
`int`	same size as uint.
`uintptr`	an unsigned integer large enough to store the uninterpreted bits of a

	pointer value
`uint8`	the set of all unsigned 8-bit integers (0 to 255)
`uint16`	the set of all unsigned 16-bit integers (0 to 65535)
`uint32`	the set of all unsigned 32-bit integers (0 to 4294967295)
`uint64`	the set of all unsigned 64-bit integers (0 to 18446744073709551615)
`int8`	the set of all signed 8-bit integers (-128 to 127)
`int16`	the set of all signed 16-bit integers (-32768 to 32767)
`int32`	the set of all signed 32-bit integers (-2147483648 to 2147483647)
`int64`	the set of all signed 64-bit integers (-9223372036854775808 to 9223372036854775807)
`float32`	the set of all IEEE-754 32-bit floating-point numbers
`float64`	the set of all IEEE-754 64-bit floating-point numbers
`complex64`	the set of all complex numbers with `float32` real and imaginary parts

`complex128`	the set of all complex numbers with `float64` real and imaginary parts
`byte`	alias for `uint8`
`rune`	alias for `int32` (represents a Unicode code point)

Example

Given below are example declarations of variables of some of the built-in types:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "math/cmplx"
)

var (
        goIsFun bool       = true //declaring a variable of type bool
        maxInt  uint64     = 1<<64 - 1 //declaring a variable of type uint64
        complex complex128 = cmplx.Sqrt(-5 + 12i) //declaring a variable of type complex128
)

func main() {
        const f = "%T(%v)\n"
        fmt.Printf(f, goIsFun, goIsFun)
        fmt.Printf(f, maxInt, maxInt)
        fmt.Printf(f, complex, complex)
}
```

Type Conversion

This lesson discusses the conversion variable types to other inbuilt types using example

WE'LL COVER THE FOLLOWING

- Converting Values
- Example

Converting Values

In the previous lesson, we covered the various types a variable in Go can take. Now, we will discuss how the type of a given variable can be changed. Converting values from one type to another is fairly simple in Go. The expression `T(v)` converts the value `v` to the type `T`.

Example

Given below are some example numeric conversions:

C++

```
#include <iostream>
using namespace std;

int main() {
  // your code goes here
  cout << "Hello World";
  return 0;
}
```

```
var i int = 42
var f float64 = float64(i)
var u uint = uint(f)
```

These can be put more simply:

```
i := 42
f := float64(i)
u := uint(f)
```

Go assignment between items of different type requires an explicit conversion which means that you manually need to convert types if you are passing a variable to a function expecting another type. Now that we know how variable types can be changed, in the following section, we will look into how you can convert a given value to another specific type.

Type Assertion

This lesson explains converting the type of a given value to another specific value using examples

WE'LL COVER THE FOLLOWING

- Introduction
- Examples
- Quiz

Introduction

In the previous lesson, we covered type conversions. Now, we will move on to discuss how, if you have a value that you want to convert to another or a specific type (in case of `interface{}`), you can use **type assertion**. A type assertion takes a value and tries to create another version in the specified explicit type.

Examples

In the example below, the `timeMap` function takes a value and if it can be asserted as a map of `interfaces{}` keyed by strings, then it injects a new entry called "updated_at" with the current time value.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "time"
)

func timeMap(y interface{}) {
        z, ok := y.(map[string]interface{}) //asserting y as a map of interfaces
```

```
        if ok {
                z["updated_at"] = time.Now() //z now has the type map[string]interface
        }

}

func main() {
        foo := map[string]interface{}{
                "Matt": 42,
        }
        timeMap(foo)
        fmt.Println(foo)
}
```

The type assertion doesn't have to be done on an empty interface. It's often used when you have a function taking a param of a specific interface but the function inner code behaves differently based on the actual object type. Here is an example:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

type Stringer interface {
        String() string
}

type fakeString struct {
        content string
}

// function used to implement the Stringe interface
func (s *fakeString) String() string {
        return s.content
}

func printString(value interface{}) {
        switch str := value.(type) {
        case string:
                fmt.Println(str)
        case Stringer:
                fmt.Println(str.String())
        }
}

func main() {
        s := &fakeString{"Ceci n'est pas un string"}
        printString(s)
```

```
        printString("Hello, Gophers")

}
```

Another example is when checking if an error is of a certain type:

```
if err != nil {
  if msqlerr, ok := err.(*mysql.MySQLError); ok && msqlerr.Number == 1062 {
    log.Println("We got a MySQL duplicate :(")
  } else {
    return err
  }
}
```

- Read more in the Effective Go guide

Quiz

Quiz on Type Assertion

Q

Which of the following is Type Assertion used for?

Check Answers

Now that we know how inbuilt types can be manipulated, we will move on to discuss how we can create our own types in Go.

Structs

This lesson explains how to define new types using Structs using an example

WE'LL COVER THE FOLLOWING

- Definition
- Examples
- Quiz

Definition

We previously discussed the in built types in Go, we will now move on to look at how new types can be defined using structs. A **struct** is a collection of fields/properties. You can define new types as structs or interfaces. If you are coming from an object-oriented background, you can think of a struct to be a light class that supports composition but not inheritance. Methods are discussed at length in Chapter 5.

You don't need to define getters and setters on struct fields, they can be accessed automatically. However, note that only exported fields (capitalized) can be accessed from outside of a package.

A struct literal sets a newly allocated struct value by listing the values of its fields. You can list just a subset of fields by using the `"Name:" syntax` (the order of named fields is irrelevant when using this syntax). The special prefix `&` constructs a pointer to a newly allocated struct.

Examples

Given below is an example struct declaration:

```
package main

import (
        "fmt"
```

```
        "time"
)

type Bootcamp struct {
        // Latitude of the event
        Lat float64
        // Longitude of the event
        Lon float64
        // Date of the event
        Date time.Time
}

func main() {
        fmt.Println(Bootcamp{
                Lat:  34.012836,
                Lon:  -118.495338,
                Date: time.Now(),
        })
}
```

Declaration of struct literals:

```
package main

import "fmt"

type Point struct {
        X, Y int
}

var (
        p = Point{1, 2}  // has type Point
        q = &Point{1, 2} // has type *Point
        r = Point{X: 1}  // Y:0 is implicit
        s = Point{}      // X:0 and Y:0
)

func main() {
        fmt.Println(p, q, r, s)
}
```

Accessing fields using the dot notation:

```
package main

import (
        "fmt"
        "time"
```

```
)

type Bootcamp struct {
        Lat, Lon float64
        Date     time.Time
}

func main() {
        event := Bootcamp{
                Lat: 34.012836,
                Lon: -118.495338,
        }
        event.Date = time.Now()
        fmt.Printf("Event on %s, location (%f, %f)",
                event.Date, event.Lat, event.Lon)

}
```

Quiz

Quiz on Structs

Q

Which of the following is NOT true regarding structs?

COMPLETED 0%

1 of 1

Now that we have learnt of all the different types in Go, we can move on to

discuss how variables can be initialized.

Initializing

This lesson discusses initializing variables using the new expression in Go

WE'LL COVER THE FOLLOWING

- Using the `new` Expression
- Resources

Using the `new` Expression

Now that we know the different types variables can take, we will look into initializing variables. Go supports the `new` expression to allocate a zeroed value of the requested type and to return a pointer to it.

```
x := new(int)
```

As seen in Section: Structs a common way to "initialize" a variable containing a struct or a reference to one, is to create a struct literal. Another option is to create a constructor. This is usually done when the zero value isn't good enough and you need to set some default field values for instance.

Note that following expressions using `new` and an empty struct literal are equivalent and result in the same kind of allocation/initialization:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
)

type Bootcamp struct {
```

```
type Bootcamp struct {
        Lat float64
        Lon float64
}

func main() {
        x := new(Bootcamp)
        y := &Bootcamp{}
        fmt.Println(*x == *y)
}
```

Note that slices, maps and channels are usually allocated using `make` so the data structure these types are built upon can be initialized.

Resources

- Allocation with `new` - effective Go
- Composite Literals - effective Go
- Allocation with `make` - effective Go

Now that we have gone over initializing variables in Go, we will look at how we can use this knowledge to implement composition, the alternative to inheritance in Go.

Composition vs Inheritance

This explains details of Composition in Go and how it can be used as an alternative to Inheritance in Go.

WE'LL COVER THE FOLLOWING

- Composition as an Alternative to Inheritance
- Composition
- Test Yourself!

Composition as an Alternative to Inheritance

Coming from an OOP background a lot of us are used to inheritance, something that isn't supported by Go. Instead you have to think in terms of *composition* and *interfaces*. In the previous lessons, we learnt about structs in Go, and that is what we will be using for composition.

The Go team wrote a short but good segment on this topic.

Composition

Composition (or embedding) is a well understood concept for most OOP programmers and Go supports it, here is an example of the problem it's solving:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

type User struct {
        Id       int
        Name     string
        Location string
```

```
}

//type Player with one additional attribute


type Player struct {
        Id       int
        Name     string
        Location string
        GameId   int
}

func main() {
        p := Player{}
        p.Id = 42
        p.Name = "Matt"
        p.Location = "LA"
        p.GameId = 90404
        fmt.Printf("%+v", p) // the value in a default format when printing structs,
                                // the plus flag (%+v) adds field names
}
```

The above example demonstrates a classic OOP challenge, our `Player` struct has the same fields as the `User` struct but it also has a `GameId` field. Having to duplicate the field names isn't a big deal, but it can be simplified by composing our struct.

Environment Variables

Key:	Value:
GOPATH	/go

```
type User struct {
        Id              int
        Name, Location string
}

type Player struct {
        User //user will contain all the required attributes
        GameId int
}
```

We can initialize a new variable of type `Player` two different ways.

Using the dot notation to set the fields:

Environment Variables

Key: Value:

GOPATH /go

```
package main

import "fmt"

type User struct {
        Id              int
        Name, Location string
}

type Player struct {
        User
        GameId int
}

func main() {
        p := Player{} //initializing
        p.Id = 42
        p.Name = "Matt"
        p.Location = "LA"
        p.GameId = 90404
        fmt.Printf("%+v", p)
}
```

The other option is to use a struct literal:

Environment Variables

Key: Value:

GOPATH /go

```
package main

import "fmt"

type User struct {
        Id              int
        Name, Location string
}

type Player struct {
        User
        GameId int
}

func main() {
        p := Player{
                User{Id: 42, Name: "Matt", Location: "LA"},
                90404,
        }
        fmt.Printf(
```

```
                "Id: %d, Name: %s, Location: %s, Game id: %d\n",
                p.Id, p.Name, p.Location, p.GameId)
        // Directly set a field defined on the Player struct

        p.Id = 11
        fmt.Printf("%+v", p)
}
```

When using a struct literal with an implicit composition, we can't just pass the composed fields. We instead need to pass the types composing the struct. Once set, the fields are directly available.

Because our struct is composed of another struct, the methods on the `User` struct is also available to the `Player`. Let's define a method to show that behavior:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

type User struct {
        Id              int
        Name, Location string
}

func (u *User) Greetings() string {
        return fmt.Sprintf("Hi %s from %s",
                u.Name, u.Location)
}

type Player struct {
        User
        GameId int
}

func main() {
        p := Player{}
        p.Id = 42
        p.Name = "Matt"
        p.Location = "LA"
        fmt.Println(p.Greetings())
}
```

As you can see this is a very powerful way to build data structures but it's even more interesting when thinking about it in the context of interfaces. By composing one of your structure with one implementing a given interface, your structure automatically implements the interface.

Here is another example, this time we will look at implementing a `Job` struct that can also behave as a logger.

Here is the explicit way:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "log"
        "os"
)

type Job struct {
        Command string
        Logger  *log.Logger
}

func main() {
        job := &Job{"demo", log.New(os.Stdout, "Job: ", log.Ldate)}
        // same as
        // job := &Job{Command: "demo",
        //             Logger: log.New(os.Stderr, "Job: ", log.Ldate)}
        job.Logger.Print("test")
}
```

Our `Job` struct has a field called `Logger` which is a pointer to another type (log.Logger)

When we initialize our value, we set the logger so we can then call its `Print` function by chaining the calls: `job.Logger.Print()`

But Go lets you go even further and use implicit composition. We can skip defining the field for our logger and now all the methods available on a

pointer to `log.Logger` are available from our struct:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "log"
        "os"
)

type Job struct {
        Command string
        *log.Logger
}

func main() {
        job := &Job{"demo", log.New(os.Stdout, "Job: ", log.Ldate)}
        job.Print("starting now...")
}
```

Note that you still need to set the logger and that's often a good reason to use a constructor (custom constructors are used when you need to set a structure before using a value, see Custom constructors. What is really nice with the implicit composition is that it allows to easily and cheaply make your structs implement interfaces. Imagine that you have a function that takes variables implementing an interface with the `Print` method. By adding `*log.Logger` to your struct (and initializing it properly), your struct is now implementing the interface without you writing any custom methods.

Test Yourself!

Quiz on Composition

Q

True or False: When using a struct literal with an implicit composition, we can pass the composed fields.

Check Answers

That concludes the lesson on how composition works. The following section contains an exercise based on this example to further enhance your understanding of the concept of composition.

Quiz on Types

A quick quiz to test understanding of this chapter!

1 When is type assertion used?

2 A struct is a collection of fields/properties?

3 What does the code below denote?

```
x := new(int)
```

4

Composition can be achieved in Go is by embedding one struct type into another?

Check Answers

Exercise on Composition

Here is an exercise you can solve to test your understanding!

WE'LL COVER THE FOLLOWING

- Question

Question

Looking at the `User` / `Player` example, you might have noticed that we composed `Player` using `User`. This means that a Player should be able to access methods defined in the `User` struct. In the code given below, add additional code to the `GreetingsForPlayer` function so that it uses the `Greetings` function from the `User` struct to print the string that the `Greetings` function is printing right now:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"
import "encoding/json"


type User struct {
        Id              int
        Name, Location string
}

func (u User) Greetings() string {
        return fmt.Sprintf("Hi %s from %s",
                     u.Name, u.Location)
}

type Player struct {
        u User
        GameId int
}
```

```
func GreetingsForPlayer(p Player) string{
  //insert code


  return ""; //modify the statement to return the required string
}
```

Working with arrays

This lesson explains arrays, multidimensional arrays and how to print them in GO

WE'LL COVER THE FOLLOWING

- Arrays
  - Printing arrays
  - Multi-dimensional arrays

Arrays

The type `[n]T` is an array of `n` values of type `T`.

The expression:

Environment Variables

Key:	Value:
GOPATH	/go

```
var a [10]int
```

declares a variable `a` as an array of ten integers.

An array's length is part of its type, so arrays *cannot* be resized. This seems limiting, but don't worry; Go provides a convenient way of working with arrays.

Let's take a look at an example below:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
    var a [2]string   //array of size 2
    a[0] = "Hello"    //Zero index of "a" has "Hello"
    a[1] = "World"    //1st index of "a" has "World"
    fmt.Println(a[0], a[1]) // will print Hello World
    fmt.Println(a)           // will print [Hello World]
}
```

Below is another example to further help understand the concept:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        var a [2]string
        a[0] = "Hello"
        a[1] = "World"
        fmt.Println(a[0], a[1])
        fmt.Println(a)

        primes := [6]int{2, 3, 5, 7, 11, 13}  // array of prime numbers of size 6
        fmt.Println(primes)
}
```

You can also set the array entries as you declare the array:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"
```

```
func main() {
    a  := [2]string{"hello", "world!"}

    fmt.Printf("%q", a)
}
```

Finally, you can use an **ellipsis** to use an implicit length when you pass the values:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        a := [...]string{"hello", "world!"}
        fmt.Printf("%q", a)
}
```

Printing arrays

Note how we used the `fmt` package using `Printf` and used the `%q` "verb" to print each element quoted.

If we had used `Println` or the `%s` verb, we would have had a different result:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        a := [2]string{"hello", "world!"}
        fmt.Println(a)
        // [hello world!]
```

```
        fmt.Printf("%s\n", a)
        // [hello world!]
        fmt.Printf("%q\n", a)
        // ["hello" "world!"]
}
```

Multi-dimensional arrays

You can also create multi-dimensional arrays:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        var a [2][3]string
        for i := 0; i < 2; i++ {
                for j := 0; j < 3; j++ {
                        a[i][j] = fmt.Sprintf("row %d - column %d", i+1, j+1)
                }
        }
        fmt.Printf("%q", a)
        // [["row 1 - column 1" "row 1 - column 2" "row 1 - column 3"]
        //  ["row 2 - column 1" "row 2 - column 2" "row 2 - column 3"]]
}
```

Trying to access or set a value at an index that doesn't exist will prevent your program from compiling, for instance, try to compile the following code:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

func main() {
    var a [2]string
    a[3] = "Hello"
```

```
}
```

Compiler will generate Error

That's because our array is of length **2** meaning that the only 2 available indexes are **0** and **1**. Trying to access index **3** results in an error that tells us that we are trying to access an index that is *out of bounds* since our array only contains 2 elements and we are trying to access the 4th element of the array.

Slices, the type that we are going to see in the next lesson is more often used, due to the fact that we don't always know in advance the length of the array we need.

Slices in Go

This section explains the concept of slices in Go and its details such as slicing, appending slices, nil slices and finding slice length

WE'LL COVER THE FOLLOWING

- Slices
- Slicing a slice
  - Syntax
  - Making slices
  - Appending to a slice
- Length
  - Nil slices
- Resources

Slices

Slices wrap arrays to give a more general, powerful, and convenient interface to sequences of data. Except for items with an explicit dimension such as *transformation matrices*, most array programming in Go is done with slices rather than simple arrays.

Slices hold references to an underlying array, and if you assign one slice to another, both refer to the same array. If a function takes a slice argument, changes it makes to the elements of the slice will be visible to the caller, analogous to passing a pointer to the underlying array.

A slice points to an array of values and also includes a length. Slices can be resized since they are just a wrapper on top of another data structure.

`[]T` is a slice with elements of type `T`.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
    p := []int{2, 3, 5, 7, 11, 13}
    fmt.Println(p)
    // [2 3 5 7 11 13]
}
```

Slicing a slice

Slices can be re-sliced, creating a new slice value that points to the same array.

Syntax

```
s[lo:hi]
```

evaluates to a slice of the elements from `lo` through `hi-1`, inclusive. Thus

```
s[lo:lo]
```

is empty and

```
s[lo:lo+1]
```

has one element.

Note: `lo` and `hi` would be integers representing indexes.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
```

```
import "fmt"

func main() {
        mySlice := []int{2, 3, 5, 7, 11, 13}
        fmt.Println(mySlice)
        // [2 3 5 7 11 13]

        fmt.Println(mySlice[1:4])
        // [3 5 7]

        // missing low index implies 0
        fmt.Println(mySlice[:3])
        // [2 3 5]

        // missing high index implies len(s)
        fmt.Println(mySlice[4:])
        // [11 13]
}
```

Let's take a look at another example below:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"

func main() {
        names := [4]string{
                "John",
                "Paul",
                "George",
                "Ringo",
        }
        fmt.Println(names)

        a := names[0:2]    //slice a
        b := names[1:3]      //slice b
        fmt.Println(a, b)

        b[0] = "XXX"      // value at zeroth index of slice b changed
        fmt.Println(a, b)
        fmt.Println(names)
}
```

In the code above two slices, `a` and `b` are made. With `a` containing elements

at the index **0** and **1** of the array and `b` containing the elements at index **1** and **2** of the array.

Making slices

Besides creating slices by passing the values right away (slice literal), you can also use `make`. You create an empty slice of a specific length and then populate each entry:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        cities := make([]string, 3)
        cities[0] = "Santa Monica"
        cities[1] = "Venice"
        cities[2] = "Los Angeles"
        fmt.Printf("%q", cities)
        // ["Santa Monica" "Venice" "Los Angeles"]
}
```

It works by allocating a zeroed array and returning a slice that refers to that array.

Appending to a slice

Note however, that you would get a runtime error if you were to do that:

```
cities := []string{}
cities[0] = "Santa Monica"
```

As explained above, a slice is seating on top of an array, in this case, the array is empty and the slice can't set a value in the referred array. There is a way to do that though, and that is by using the `append` function:

```
package main

import "fmt"
```

```
func main() {
        cities := []string{}
        cities = append(cities, "San Diego")
        fmt.Println(cities)
        // [San Diego]
}
```

You can append more than one entry to a slice:

```
package main

import "fmt"

func main() {
        cities := []string{}
        cities = append(cities, "San Diego", "Mountain View")
        fmt.Printf("%q", cities)
        // ["San Diego" "Mountain View"]
}
```

And you can also append a slice to another using an ellipsis:

```
package main

import "fmt"

func main() {
        cities := []string{"San Diego", "Mountain View"}
        otherCities := []string{"Santa Monica", "Venice"}
        cities = append(cities, otherCities...)
        fmt.Printf("%q", cities)
        // ["San Diego" "Mountain View" "Santa Monica" "Venice"]
}
```

Note that the ellipsis is a built-in feature of the language that means that the element is a collection. We can't append an element of type slice of strings (`[]string`) to a slice of strings, only strings can be appended. However, using the ellipsis (`...`) after our slice, we indicate that we want to append each element of our slice. Because we are appending strings from another slice, the

compiler will accept the operation since the types are matching.

You obviously can't append a slice of type `[]int` to another slice of type `[]string`.

Length

At any time, you can check the length of a slice by using `len`:

```
package main

import "fmt"

func main() {
        cities := []string{
                "Santa Monica",
                "San Diego",
                "San Francisco",
        }
        fmt.Println(len(cities))
        // 3
        countries := make([]string, 42)
        fmt.Println(len(countries))
        // 42
}
```

Nil slices

The zero value of a slice is nil. A nil slice has a *length* and *capacity* of **0**.

```
package main

import "fmt"

func main() {
    var z []int
    fmt.Println(z, len(z), cap(z))
    // [] 0 0
    if z == nil {
        fmt.Println("nil!")
    }
    // nil!
}
```

Resources

Resources

For more details about slices read the links below:

- Go slices, usage and internals
- Effective Go - slices
- Append function documentation
- Slice tricks
- Effective Go - slices
- Effective Go - two-dimensional slices
- Go by example - slices

Next lesson discusses *range* form of for loops for iterating over slices. Read on to find out more!

Range in for loops

This lesson discusses range form of for loops and their use to iterate slices in Go

WE'LL COVER THE FOLLOWING

- Range
- Break & continue
- Range and maps

Range

The `range` form of the for loop iterates over a slice or a map. Being able to iterate over all the elements of a data structure is very useful and `range` simplifies the iteration.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

var pow = []int{1, 2, 4, 8, 16, 32, 64, 128}

func main() {
    for i, v := range pow { //loops over the length of pow
        fmt.Printf("2**%d = %d\n", i, v)
    }
}
```

You can skip the index or value by assigning to `_`. If you only want the index, drop the “, value” entirely.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
    pow := make([]int, 10)
    for i := range pow {
        pow[i] = 1 << uint(i)
    }
    for _, value := range pow {
        fmt.Printf("%d\n", value)
    }
}
```

Break & continue

As if you were using a normal for loop, you can stop the iteration anytime by using `break`:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        pow := make([]int, 10)
        for i := range pow {
                pow[i] = 1 << uint(i)
                if pow[i] >= 16 {
                        break //stop iterating over pow when it reaches 16
                }
        }
        fmt.Println(pow)
        // [1 2 4 8 16 0 0 0 0 0]
}
```

You can also skip an iteration by using `continue`:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
	pow := make([]int, 10)
	for i := range pow {
		if i%2 == 0 {
			continue //skip each even index of pow
		}
		pow[i] = 1 << uint(i)
	}
	fmt.Println(pow)
	// [0 2 0 8 0 32 0 128 0 512]
}
```

Range and maps

Range can also be used on maps, another commonly used data structure. (discussed in the following lesson) In that case, the first parameter isn't an incremental integer but the map key:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
	cities := map[string]int{
		"New York":    8336697,
		"Los Angeles": 3857799,
		"Chicago":     2714856,
	}
	for key, value := range cities { //for each key-value pair in cities
		fmt.Printf("%s has %d inhabitants\n", key, value)
	}
}
```

In the code above `map` takes as input the `string` type, in this case, the name of cities, and maps them to integers (a concept explained in the following lesson). The loop iterates over each `key`, `value` pair in cities as `range cities` goes from **zero** to the size of **cities**.

In the next lesson, we will discuss the interesting concept of *maps* in Go.

Maps in Go

This lesson gives an introduction to maps, using the map literals and mutating maps in Go

WE'LL COVER THE FOLLOWING

- Introduction
- Mutating maps
- Resources

Introduction

Maps are somewhat similar to what other languages call “dictionaries” or “hashes”.

A map *maps* keys to values. Here we are mapping string keys (actor names) to an integer value (age).

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        celebs := map[string]int{ //mapping strings to integers
                "Nicolas Cage":       50,
                "Selena Gomez":       21,
                "Jude Law":           41,
                "Scarlett Johansson": 29,
        }

        fmt.Printf("%#v", celebs)
}
```

When not using map literals like above, maps must be created with make (not new) before use. The nil map is empty and cannot be assigned to.

Assignments follow the Go convention and can be observed in the example below.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

type Vertex struct {
        Lat, Long float64
}

var m map[string]Vertex

func main() {
        m = make(map[string]Vertex)
        m["Bell Labs"] = Vertex{40.68433, -74.39967} //assignment
        fmt.Println(m["Bell Labs"])
}
```

In the example above the `map` `m` takes `string` as input and maps it to `vertex` which is a `struct` containing two variables of type `float64`. Hence, the string “Bell” gets mapped to the value “40.68433” and “Labs” gets mapped to “-74.39967”.

When using map literals, if the top-level type is just a type name, you can omit it from the elements of the literal.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
```

```
import "fmt"

type Vertex struct {
	Lat, Long float64
}

var m = map[string]Vertex{
	"Bell Labs": {40.68433, -74.39967},
	// same as "Bell Labs": Vertex{40.68433, -74.39967}
	"Google": {37.42202, -122.08408},
}

func main() {
	fmt.Println(m)
}
```

Mutating maps

Insert or update an element in map m:

```
m[key] = elem
```

Retrieve an element:

```
elem = m[key]
```

Delete an element:

```
delete(m, key)
```

Test that a key is present with a two-value assignment:

```
elem, ok = m[key]
```

Let's take a look at an example now:

Environment Variables

Key:	Value:
GOPATH	/go

```go
package main

import "fmt"


type Vertex struct {
        Lat, Long float64
}

var m = map[string]Vertex{
        "Bell Labs": {40.68433, -74.39967},
        // same as "Bell Labs": Vertex{40.68433, -74.39967}
        "Google": {37.42202, -122.08408},
}

func main() {
  m["Splice"] = Vertex{34.05641, -118.48175} //inserting a new (key,value) here
        fmt.Println(m["Splice"])
        delete(m, "Splice")    //deleting the element
        fmt.Printf("%v\n", m)
        name, ok := m["Splice"]      //checks to see if element is present
        fmt.Printf("key 'Splice' is present?: %t - value: %v\n", ok, name)
        name, ok = m["Google"]
        fmt.Printf("key 'Google' is present?: %t - value: %v\n", ok, name)
}
```

If `key` is in `m`, `ok` is true. If not, `ok` is false and `elem` is the zero value for the map's element type. Similarly, when reading from a map if the key is not present the result is the zero value for the map's element type.

Resources

- Go team blog post on maps
- Effective Go - maps

This marks the end of this chapter. Read on to the next one to learn more interesting concepts in Go.

Quiz on Collection Types

Q

True or False: arrays cannot be resized in Go.

Check Answers

Slice

Q

Given a slice, s, which of the following would result in an error?

Check Answers

Q

Which of the following allows you to skip an iteration of a for loop?

Check Answers

Maps

Q

True or False: Maps are created with the new keyword in Go.

Check Answers

Exercise on Maps

Here is an exercise you can solve to test your understanding!

WE'LL COVER THE FOLLOWING

- Implement WordCount
- Question

Implement WordCount

Have a look at this Online assignment to understand maps better before attempting the question below.

Question

The code below should return a map of the counts of each "word" in the string `s`.

You might find strings.Fields helpful in order to solve this question.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
   _"strings"
   "fmt"
   "encoding/json"
   "strconv"
)

func WordCount(s string) map[string]int {
   // write your solution here

  return map[string]int{} //returning an empty map, modify the statement to return the correc
}
```

IF Statement

This lesson explains how to use if statements in Go using examples

WE'LL COVER THE FOLLOWING

- Comparison With Other Languages
- Example

Comparison With Other Languages

The `if` statement looks as it does in C or Java, except that the `( )` are gone and the `{ }` are required. Like `for`, the `if` statement can start with a short statement to execute before the condition. Variables declared by the statement are only in scope until the end of the `if`. Variables declared inside an `if` short statement are also available inside any of the `else` blocks.

- Example of an `if` statement:

```
if answer != 42 {
    return "Wrong answer"
}
```

Here's a complete example to help you better understand the concept:

```
package main

import (
        "fmt"
        "math"
)

func sqrt(x float64) string {
        if x < 0 {
                return sqrt(-x) + "i"
        }
        return fmt.Sprint(math.Sqrt(x))
}

func main() {
```

```
func main() {
        fmt.Println(sqrt(2), sqrt(-4))
}
```

- Example of an `if` short statement with a variable declared within the statement:

```
if err := foo(); err != nil {
    panic(err)
}
```

Here's the complete example using this concept:

```
package main

import (
        "fmt"
        "math"
)

func pow(x, n, lim float64) float64 {
        if v := math.Pow(x, n); v < lim {
                return v
        }
        return lim
}

func main() {
        fmt.Println(
                pow(3, 2, 10),
                pow(3, 3, 20),
        )
}
```

Example

- Example of an `if` and `else` block:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
```

```
import (
	"fmt"

	"math"
)

func pow(x, n, lim float64) float64 {
	if v := math.Pow(x, n); v < lim {
		return v
	} else {
		fmt.Printf("%g >= %g\n", v, lim)
	}
	// can't use v here, though
	return lim
}

func main() {
	fmt.Println(
		pow(3, 2, 10),
		pow(3, 3, 20),
	)
}
```

Now that we've seen the usage of `if` statements in Go, we will move on to discuss `for` loops in Go in the next lesson.

FOR Loop

This lesson discusses for loops, including, for loop without a statement using examples and their use as an alternative to the while loop in Go

WE'LL COVER THE FOLLOWING

- Comparison with Other Languages
- `for` Loop Syntax
- For Loops Without Statements
- For loop as an Alternative to While
- Infinite Loops

Comparison with Other Languages

Go has only one looping construct, the for loop. The basic for loop looks as it does in C or Java, except that the `( )` are gone (they are not even optional) and the `{ }` are required. As in C or Java, you can leave the pre and post statements empty.

`for` Loop Syntax

- Example of a `for` loop:

```
package main

import "fmt"

func main() {
        sum := 0
        for i := 0; i < 10; i++ {
                sum += i
        }
        fmt.Println(sum)
}
```

For Loops Without Statements

- Example of a `for` loop without pre/post statements:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        sum := 1
        for ; sum < 1000; { //iterate as long as sum<1000
                sum += sum
        }
        fmt.Println(sum)
}
```

For loop as an Alternative to While

- Using a `for` loop like a `while` loop is used:

```
package main

import "fmt"

func main() {
        sum := 1
        for sum < 1000 {
                sum += sum
        }
        fmt.Println(sum)
}
```

Infinite Loops

- Infinite `for` loop:

```
package main

func main() {
```

```
        for {
  // do something in a loop forever


  }
}
```

Now that we have covered the usage of `for` loop in Go, we will discuss another popular control flow feature, switch statements.

Switch Case Statement

This lesson explains the key features of switch case statements and their use as an alternative to multiple if-else statements

WE'LL COVER THE FOLLOWING

- Alternative to Multiple `ifelse` Statements
- Key Features

Alternative to Multiple `ifelse` Statements

We covered `if else` statements in the previous lesson. However, most programming languages also have some sort of switch case statement to allow developers to avoid doing complex and ugly series of `if else` statements.

Here's an example demonstrating the concept:

```
package main

import (
        "fmt"
        "time"
)

func main() {
        now := time.Now().Unix()
        mins := now % 2
        switch mins {
        case 0:
                fmt.Println("even")
        case 1:
                fmt.Println("odd")
        }
}
```

Key Features

There are a few interesting things to know about this statement in Go:

- You can only compare values of the same type.
- You can set an optional `default` statement to be executed if all the others fail.
- You can use an expression in the `case` statement, for instance you can calculate a value to use in the case:

```
package main

import "fmt"

func main() {
	num := 3
	v := num % 2
	switch v {
	case 0:
		fmt.Println("even")
	case 3 - 2:
		fmt.Println("odd")
	}
}
```

- You can have multiple values in a `case` statement:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
	score := 7
	switch score {
	case 0, 1, 3:
		fmt.Println("Terrible")
	case 4, 5:
		fmt.Println("Mediocre")
	case 6, 7:
		fmt.Println("Not bad")
	case 8, 9:
		fmt.Println("Almost perfect")
	case 10:
```

```
                fmt.Println("hmm did you cheat?")
        default: //to be executed if no other case matches
                fmt.Println(score, " off the chart")

        }
}
```

- You can execute all the following statements after a match using the `fallthrough` statement:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        n := 4
        switch n {
        case 0:
                fmt.Println("is zero")
                fallthrough //if case matches, all following conditions will be executed as w
        case 1:
                fmt.Println("is <= 1")
                fallthrough
        case 2:
                fmt.Println("is <= 2")
                fallthrough
        case 3:
                fmt.Println("is <= 3")
                fallthrough
        case 4:
                fmt.Println("is <= 4")
                fallthrough
        case 5:
                fmt.Println("is <= 5")
                fallthrough
        case 6:
                fmt.Println("is <= 6")
                fallthrough
        case 7:
                fmt.Println("is <= 7")
                fallthrough
        case 8:
                fmt.Println("is <= 8")
                fallthrough
        default:
                fmt.Println("Try again!")
        }
}
```

You can use a `break` statement inside your matched statement to exit the switch processing:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
	"fmt"
	"time"
)

func main() {
	n := 1
	switch n {
	case 0:
		fmt.Println("is zero")
		fallthrough
	case 1:
		fmt.Println("<= 1")
		fallthrough
	case 2:
		fmt.Println("<= 2")
		fallthrough
	case 3:
		fmt.Println("<= 3")
		if time.Now().Unix()%2 == 0 {
			fmt.Println("un pasito pa lante maria")
			break //execution stops here if this case matches i.e. no other case
		}
		fallthrough
	case 4:
		fmt.Println("<= 4")
		fallthrough
	case 5:
		fmt.Println("<= 5")
	}
}
```

This concludes the discussion on all the available control flow features in Go. The following lesson contains an exercise that tests a combination of these control flow features.

Exercise on For Loops

WE'LL COVER THE FOLLOWING

- Question

Question

We have learnt that `for` loops are used as `while` loops in Go. Given below is a C++ code that uses a `while` loop to sum an array of numbers. Rewrite the code in Go so that it uses a `for` loop to sum a given list of numbers:

Environment Variables

Key:	Value:
GOPATH	/go

```
#include <iostream>
using namespace std;

int main() {
  int foo [5] = {6, 2, 77, 4, 12};
  int count=0;
  int sum=0;
  while (count<5){
    sum+=foo[count];
    count++;
  }
  cout<<"The sum is: "<<sum<<endl;
  return 0;
}
```

Given below is some starter code:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
import "fmt"
import "strconv"
import "encoding/json"

func GetSum(array [] int) int {
  //write code here

  return 0; //return the sum here
}
```

Quiz on Control Flow

Quiz on control flow features

IF statements

Q

True or False: Variables declared by an if statement are only in scope until the end of the if and its corresponding else.

COMPLETED 0%

1 of 1

FOR Loops

Q

True or False: While loops can also be used as an alternative to for loops in Go.

Switch Case Statements

Q

Which of the following statements allow all the following statements after a match to be executed in a switch case block?

Exercise on Control Flow

Here is an exercise you can solve to test your understanding!

WE'LL COVER THE FOLLOWING

- Question

Question

You have 50 bitcoins to distribute to 10 users: Matthew, Sarah, Augustus, Heidi, Emilie, Peter, Giana, Adriano, Aaron, Elizabeth The coins will be distributed based on the vowels contained in each name where:

a: 1 coin e: 1 coin i: 2 coins o: 3 coins u: 4 coins

and a user can't get more than 10 coins. Print a map with each user's name and the amount of coins distributed. After distributing all the coins, you should have 2 coins left.

The output should look something like that:

```
map[Matthew:2 Peter:2 Giana:4 Adriano:7 Elizabeth:5 Sarah:2 Augustus:10 Heidi:5 Emilie:6 Aaro
Coins left: 2
```

Note that Go doesn't keep the order of the keys in a map, so your results might not look exactly the same but the key/value mapping should be the same.

Here is some starting code:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main
```

```
import "fmt"
import "strconv"
import "encoding/json"

var (
        coins = 50
        users = []string{
                "Matthew", "Sarah", "Augustus", "Heidi", "Emilie",
                "Peter", "Giana", "Adriano", "Aaron", "Elizabeth",
        }
        distribution = make(map[string]int, len(users))
)

func GetResult([] string) string {
 //insert code here

  fmt.Println(distribution)
        fmt.Println("Coins left:", coins)
  return strconv.Itoa(distribution["Matthew"])+" "+strconv.Itoa(distribution["Sarah"])+" "+st
}
```

Introduction

This lesson explains how methods and interfaces in Go work using an example

WE'LL COVER THE FOLLOWING

- Methods in Go
- Example

Methods in Go

While technically Go isn't an Object Oriented Programming language, types and methods allow for an object-oriented style of programming. The big difference is that Go does not support type inheritance (mechanism of acquiring the features and behaviours of a class by another class) but instead has a concept of interface.

In this chapter, we will focus on Go's use of methods and interfaces.

> Note: A frequently asked question is "what is the difference between a function and a method". A method is a function that has a defined receiver, in OOP terms, a method is a function on an instance of an object.

Go does **not** have classes. However, you can define methods on struct types.

The *method receiver* appears in its own argument list between the `func` keyword and the method name. Here is an example with a `User` struct containing two fields: `FirstName` and `LastName` of string type.

Example

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
)

type User struct {
        FirstName, LastName string
}

func (u User) Greeting() string {
        return fmt.Sprintf("Dear %s %s", u.FirstName, u.LastName)
}

func main() {
        u := User{"Matt", "Aimonetti"}
        fmt.Println(u.Greeting())
}
```

Note how methods are defined outside of the struct, if you have been writing Object Oriented code for a while, you might find that a bit odd at first. The method on the `User` type could be defined anywhere in the package.

Now that we know how methods and interfaces in Go work, in the following lesson, we will look into how the code itself should be organized.

Code Organization

This lesson explains how to organize code in Go using an example

WE'LL COVER THE FOLLOWING

- Example

Example

Methods can be defined on any file in the package, but my recommendation is to organize the code as shown below:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

// list of packages to import
import (
        "fmt"
)

// list of constants
const (
        ConstExample = "const before vars"
)

// list of variables
var (
        ExportedVar    = 42
        nonExportedVar = "so say we all"
)

// Main type(s) for the file,
// try to keep the lowest amount of structs per file when possible.
type User struct {
        FirstName, LastName string
        Location            *UserLocation
}

type UserLocation struct {
```

```
        City    string
        Country string
}


// List of functions
func NewUser(firstName, lastName string) *User {
        return &User{FirstName: firstName,
                LastName: lastName,
                Location: &UserLocation{
                        City:    "Santa Monica",
                        Country: "USA",
                },
        }
}

// List of methods
func (u *User) Greeting() string {
        return fmt.Sprintf("Dear %s %s", u.FirstName, u.LastName)
}

func main() {
  us:=User {FirstName: "Matt",
                LastName: "Damon",
                Location: &UserLocation{
                        City:    "Santa Monica",
                        Country: "USA",}}
  fmt.Println(us.Greeting())
}
```

In fact, you can define a method on `any` type you define in your package, not just structs. You cannot define a method on a type from another package, or on a basic type.

In the next lesson, we will take a look at *type aliasing*.

Type Aliasing

This lesson takes a look at the concept of aliasing using an example

WE'LL COVER THE FOLLOWING

- Example

To define methods on a type you don't "own", you need to define an *alias* for the type you want to extend.

Example

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "strings"
)

type MyStr string //using MyStr as an alias for type string

func (s MyStr) Uppercase() string {
        return strings.ToUpper(string(s))
}

func main() {
        fmt.Println(MyStr("test").Uppercase())
}
```

Down below is another example explaining the concept of aliasing for better understanding.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
    "fmt"
    "math"
)

type MyFloat float64   //using MyFloat as an alias for type float64

func (f MyFloat) Abs() float64 {
    if f < 0 {
        return float64(-f)
    }
    return float64(f)
}

func main() {
    f := MyFloat(-math.Sqrt2)
    fmt.Println(f.Abs())
}
```

As you can see above, type aliasing declares a new name to be used as a substitute for a previously known type. It does not introduce a new type, neither does it change the meaning of the existing type name.

In the next lesson, we will discuss *method receivers*.

Method Receivers

This lesson explains with examples method receivers in Go

WE'LL COVER THE FOLLOWING

- Example

Methods can be associated with a *named* type (`User` for instance) or a *pointer* to a named type (`*User`). In the two type aliasing examples in previous lesson, methods were defined on the value types (`MyStr` and `MyFloat`).

Example

There are two reasons to use a *pointer receiver*. First, to avoid copying the value on each method call (more efficient if the value type is a large struct). The previous `User` example would have been better written as follows:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
)

type User struct {
        FirstName, LastName string
}

func (u *User) Greeting() string { //pointers
        return fmt.Sprintf("Dear %s %s", u.FirstName, u.LastName)
}

func main() {
        u := &User{"Matt", "Aimonetti"}
        fmt.Println(u.Greeting())
}
```

Remember that Go passes everything by value, meaning that when `Greeting()` is defined on the value type, every time you call `Greeting()`, you are copying the `User` struct. Instead when using a pointer, only the pointer is copied (cheap).

The other reason why you might want to use a pointer is so that the method can modify the value that its receiver points to.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "math"
)

type Vertex struct {
        X, Y float64
}

func (v *Vertex) Scale(f float64) {
        v.X = v.X * f //v will be modified directly here
        v.Y = v.Y * f
}

func (v *Vertex) Abs() float64 {
        return math.Sqrt(v.X*v.X + v.Y*v.Y)
}

func main() {
        v := &Vertex{3, 4}
        v.Scale(5)
        fmt.Println(v, v.Abs())
}
```

In the example above, `Abs()` could be defined on the value type or the pointer since the method doesn't modify the receiver value (the vertex). However `Scale()` has to be defined on a pointer since it does modify the receiver. `Scale()` resets the values of the `X` and `Y` fields.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "math"
)

type MyFloat float64

func (f MyFloat) Abs() float64 {
        if f < 0 {
                return float64(-f)
        }
        return float64(f)
}

func main() {
        f := MyFloat(-math.Sqrt2)
        fmt.Println(f.Abs())
}
```

This marks the end of this chapter. In the next chapter, we will discuss *interfaces*. Read on to find out more!

Quiz on Go Methods

A quick quiz to test understanding of this chapter!

1

A method is a function on an instance of an object

2

Type aliasing:

Check Answers

Introduction

This lesson gives an introduction to interfaces in Go using an example, also explains how to define a new type that would implement the same interface.

WE'LL COVER THE FOLLOWING

- Definition
- Example
- Defining a New Type

Definition

An **interface** type is defined by a set of methods. A value of interface type can hold any value that implements those methods. Interfaces increase the flexibility as well as the scalability of the code. Hence, it can be used to achieve polymorphism in Golang. Interface does not require a particular type, specifying that only some behavior is needed, which is defined by a set of methods.

Example

Here is a refactored version of our earlier example. This time we made the greeting feature more generic by defining a function called `Greet` which takes a param of interface type `Namer`. `Namer` is a new interface we defined which only defines one method: `Name()`. So `Greet()` will accept as param any value which has a `Name()` method defined.

To make our `User` struct implement the interface, we defined a `Name()` method. We can now call `Greet` and pass our pointer to `User` type.

Environment Variables

Key:	Value:
GOPATH	/go

GOPATH /go

```
package main

import (
        "fmt"
)

type User struct {
        FirstName, LastName string
}

func (u *User) Name() string {
        return fmt.Sprintf("%s %s", u.FirstName, u.LastName)
}

type Namer interface {
  Name() string //The Namer interface is defined by the Name() method
}

func Greet(n Namer) string {
        return fmt.Sprintf("Dear %s", n.Name())
}

func main() {
        u := &User{"Matt", "Aimonetti"}
        fmt.Println(Greet(u))
}
```

Defining a New Type

We could now define a new type that would implement the same interface and our `Greet` function would still work.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
)

type User struct {
        FirstName, LastName string
}

func (u *User) Name() string { //Name method used for type User
        return fmt.Sprintf("%s %s", u.FirstName, u.LastName)
}

type Customer struct {
```

```
type Customer struct {
        Id       int
        FullName string
}

func (c *Customer) Name() string { //Name method used for type Customer
        return c.FullName
}

type Namer interface {
  Name() string //Both Name() methods can be called using the Namer interface
}

func Greet(n Namer) string {
        return fmt.Sprintf("Dear %s", n.Name())
}

func main() {
        u := &User{"Matt", "Aimonetti"}
        fmt.Println(Greet(u))
        c := &Customer{42, "Francesc"}
        fmt.Println(Greet(c))
}
```

Now that we have a basic understanding of interfaces in Go, we will continue to look into how they are satisfied in the next section.

Satisfying Interfaces

This lesson discusses implicit interfaces and how Go interfaces are satisfied.

WE'LL COVER THE FOLLOWING

- Implicit Interfaces

Implicit Interfaces

A type implements an interface by implementing the methods that it contains.

There is no explicit declaration of intent. The interfaces are satisfied implicitly through its methods.

Implicit interfaces decouple implementation packages from the packages that define the interfaces: neither depends on the other.

It also encourages the definition of **precise interfaces**, because you don't have to find every implementation and tag it with the new interface name.

Given below is an example implementation of Reader and Writer interfaces in Go:

```
package main

import (
        "fmt"
        "os"
)

type Reader interface {
        Read(b []byte) (n int, err error)
}

type Writer interface {
        Write(b []byte) (n int, err error)
}

type ReadWriter interface {
        Reader
```

```
        Writer
}


func main() {
        var w Writer

        // os.Stdout implements Writer
        w = os.Stdout

        fmt.Fprintf(w, "hello, writer\n")
}
```

Package io has Reader and Writer defined already so you don't have to.

Now that we have a clear idea of how interfaces in Go are implemented, we will look into how error messages can be displayed within them in the following lesson.

Returning Errors

This lesson talks about Displaying error messages in Go and use of fmt package for printing

WE'LL COVER THE FOLLOWING

- Returning an Error Message
- The `fmt` Package

Returning an Error Message

In the previous lessons, we learned how to implement interfaces in Go. We will now look at how we can display error messages within them. An **error** is anything that can describe itself as an error string. The idea is captured by the predefined, built-in interface type, error, with its single method, `Error`, returning a string:

```
type error interface {
    Error() string
}
```

The `fmt` Package

The `fmt` package's various print routines automatically know to call the method when asked to print an error.

```
package main

import (
    "fmt"
    "time"
)

type MyError struct {
    When time.Time
    What string
}

func (e *MyError) Error() string {
```

```
    return fmt.Sprintf("at %v, %s",
        e.When, e.What)
}


func run() error {
    return &MyError{
        time.Now(),
        "it didn't work",
    }
}

func main() {
    if err := run(); err != nil {
        fmt.Println(err)
    }
}
```

To better enhance your understanding of this concept, an exercise on errors is provided at the end of the chapter.

Quiz on Go Interfaces

Here's a quick quiz to test your understanding of Interfaces!

Interfaces

1

True or False: An interface type is defined by a set of methods, which means that two types that have different names, but the same method names can use the same interface.

2

Which of the following is NOT a benefit of implicit interfaces?

Check Answers

Errors

Q

Which package is used to automatically print an error message?

COMPLETED 0%

1 of 1

Exercise on Errors

Here is an exercise you can solve to test your understanding!

WE'LL COVER THE FOLLOWING

- Assignment

Assignment

Copy your `Sqrt` function from the earlier exercises and modify it to return an `error` value. (This online assignment may be useful)

`Sqrt` should return a non-nil error value when given a negative number, as it doesn't support complex numbers.

Create a new type

```
type ErrNegativeSqrt float64
```

and make it an error by giving it a

```
func (e ErrNegativeSqrt) Error() string
```

method such that `ErrNegativeSqrt(-2).Error()` returns

`cannot Sqrt negative number: -2`.

Note: a call to `fmt.Print(e)` inside the Error method will send the program into an infinite loop. You can avoid this by converting e first: `fmt.Print(float64(e))`. Why?

Change your `Sqrt` function to return an `ErrNegativeSqrt` value when given a negative number.

Given below is some starter code, add your code to implement the required function:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
  "strconv"
  "encoding/json"
)

type ErrNegativeSqrt float64

func (e ErrNegativeSqrt) Error() string {
        return fmt.Sprintf("cannot Sqrt negative number")
}

//Your function should return nil for error when the number can be squared
//When the number cannot be squared, the function should return 0 along with the error
func Sqrt(x float64) (float64, error) {
        //write code here

  return -1, nil //edit to return the correct answer
}
```

Tip: When doing an inferred declaration of a float, you can omit the decimal value and do the following:

```
z := 1.
// same as
// z := 1.0
```

Concurrent Programming

This lesson introduces the concept of concurrent programming

WE'LL COVER THE FOLLOWING

- Introduction

Introduction

Concurrent programming is a large topic, but it's also one of the most interesting aspects of the Go language.

Concurrent programming in many environments is made difficult by the subtleties required to implement correct access to *shared variables*. Go encourages a different approach in which shared values are passed around on channels and, in fact, never actively shared by separate threads of execution. Only one goroutine has access to the value at any given time.

Data races cannot occur, by design. To encourage this way of thinking we have reduced it to a slogan:

> Do not communicate by sharing memory; instead, share memory by communicating.

This approach can be taken too far. Reference counts may be best done by putting a *mutex* around an integer variable, for instance. But as a high-level approach, using channels to control access makes it easier to write clear, correct programs.

Although Go's approach to concurrency originates in Hoare's Communicating Sequential Processes (CSP), it can also be seen as a type-safe generalization of Unix pipes.

- Rob Pike's concurrency slides (IO 2012)
- Video of Rob Pike at IO 2012
- Video of Concurrency is not parallelism (Rob Pike)

In the next lesson, we'll introduce you to the interesting concept of *goroutines*, so keep on reading!

Goroutines

This lesson introduces the concept of goroutines, their definition, and gives an example.

WE'LL COVER THE FOLLOWING

- Definition
- Example

Definition

A goroutine is a lightweight thread managed by the Go runtime. Goroutines can be functions or methods that run concurrently with other functions or methods

```
go f(x, y, z)
```

starts a new goroutine running:

```
f(x, y, z)
```

The evaluation of `f`, `x`, `y`, and `z` happens in the current goroutine, and the execution of `f` happens in the new goroutine.

Goroutines run in the same address space, so access to shared memory must be **synchronized**. The sync package provides useful primitives, although you won't need them much in Go as there are other primitives.

Example

Environment Variables

Key: Value:

GOPATH	/go

```
package main

import (
        "fmt"
        "time"
)

func say(s string) {
        for i := 0; i < 5; i++ {
                time.Sleep(100 * time.Millisecond)
                fmt.Println(s)
        }
}

func main() {
  go say("world") //parallel execution: both calls to say() will execute concurrently
        say("hello")
}
```

The example above is a good representation of how *goroutine* works. As you can see from the output, “world” isn’t printed first even though line number **16** is written first in the code; this is because it executes in parallel to the next line which will print “hello”.

In the next lesson, we will discuss *channels* used in Go. Keep on reading to find out more!

Channels

This lesson explains in detail the workings of channels in Go

WE'LL COVER THE FOLLOWING

- About channels
- Example
- Buffered channels

About channels

Channels are pipes through which you can send and receive values using the channel operator, `<-`.

Environment Variables

Key:	Value:
GOPATH	/go

```
ch <- v    // Send v to channel ch.
v := <-ch  // Receive from ch, and
           // assign value to v.
```

(The data flows in the direction of the arrow.)

Like maps and slices, channels must be created before use:

```
ch := make(chan int)
```

By default, sends and receives block wait until the other side is ready. This allows goroutines to synchronize without explicit locks or condition variables.

Example

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func sum(a []int, c chan int) {
        sum := 0
        for _, v := range a {
                sum += v
        }
        c <- sum // send sum to c
}

func main() {
        a := []int{7, 2, 8, -9, 4, 0}

        c := make(chan int)
        go sum(a[:len(a)/2], c)
        go sum(a[len(a)/2:], c)
        x, y := <-c, <-c // receive from c

        fmt.Println(x, y, x+y)
}
```

In the example above, all lines before line **17** will execute sequentially. Line **17** will invoke a non-blocking go-routine for the `sum` function and will pass the channel `c` and the first half of the array in it. As the code does not block, the function invoked by line 17 and line 18 of the main program executes parallelly. Line **18** will take the second half of the array and the channel `c` as inputs.

Hence, both the go-routines invoked in line **17** and **18** will execute in parallel.

Line **19** will not execute until we receive a value from the channels. When the go-routines invoked by line **17** and **18** will pass values to the channels (as specified by `c <- sum`), line **19** of the main will then execute.

Buffered channels

Channels can be *buffered*. Provide the buffer length as the second argument to `make` to initialize a buffered channel:

```
ch := make(chan int, 100)
```

Sends to a buffered channel block only when the buffer is full. Receives block when the buffer is empty.

```
package main

import "fmt"

func main() {
    c := make(chan int, 2)
    c <- 1
    c <- 2
    fmt.Println(<-c)
    fmt.Println(<-c)
}
```

But if you do:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        c := make(chan int, 2)
        c <- 1
        c <- 2
        c <- 3
        fmt.Println(<-c)
        fmt.Println(<-c)
        fmt.Println(<-c)
}
```

You are getting a deadlock!

That's because we overfilled the buffer without letting the code a chance to read/remove a value from the channel.

However, this version using a goroutine would work fine:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func main() {
        c := make(chan int, 2)
        c <- 1
        c <- 2
        c3 := func() { c <- 3 }
        go c3()
        fmt.Println(<-c)
        fmt.Println(<-c)
    fmt.Println(<-c)
}
```

The reason is that we are adding an extra value from inside a go routine, so our code doesn't block the main thread. The goroutine is being called before the channel is being emptied, but that is fine, the goroutine will wait until the channel is available. We then read the first value from the channel, which frees a spot and our goroutine can push its value to the channel.

Click on Go tour page to test out the example below yourself by modifying it.

```
package main

import "fmt"

func main() {
        ch := make(chan int, 2)
        ch <- 1
        ch <- 2
        fmt.Println(<-ch)
        fmt.Println(<-ch)
}
```

Having trouble visualizing channels? No worries, the next lesson explains

channels through illustrations to help you better understand the concept.

Channels Through Illustrations

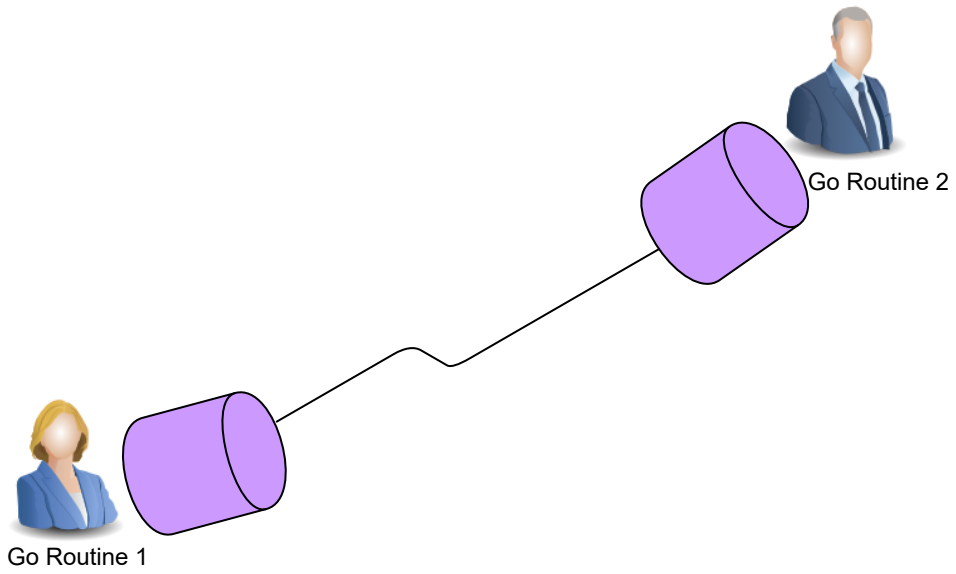
Explaining Channels in Go through illustrations

WE'LL COVER THE FOLLOWING

- Channels
- Channels as Pipes
- Sending and Receiving
- Blocking on a Send
- Blocking on a Receive

Channels

Channels are essentially means through which Go routines communicate with one another.

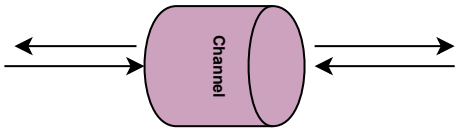


Channels in Go

Channels as Pipes

A channel may be visualized as a pipe, through which go routines can send and receive information from other Go routines.

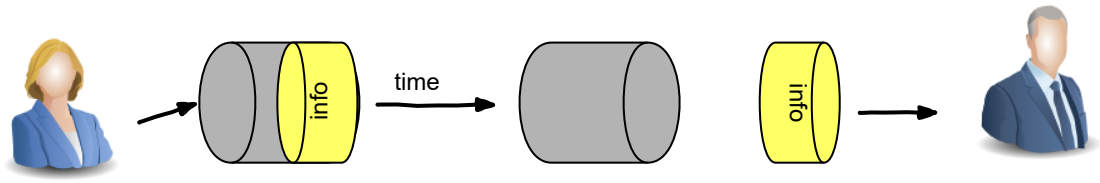
and receive information from other Go routines.



Go Channels as Pipes

Sending and Receiving

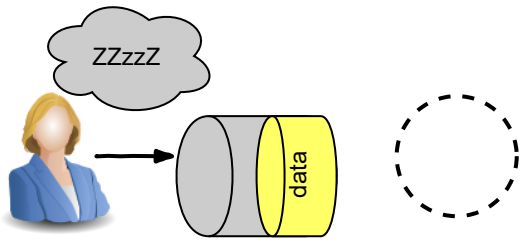
Go routines can *send* and *receive* on a channel. This is done through using an arrow (<-) that points in the direction that the data is going.



How Go Routines Communicate Through Channels

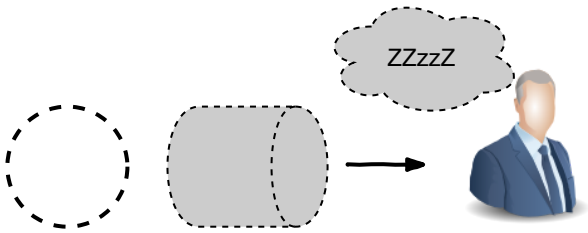
Blocking on a Send

Once a Go routine sends information on a channel, the sending Go routine blocks until another Go routine receives what was sent on the channel.



Blocking on a Receive

Similar to blocking after sending on a channel, a Go routine can block waiting to get a value from a channel, with nothing sent to it yet.



Now that we've covered the basics of Channels, the following lesson will illustrate the differences between buffered and unbuffered channels.

illustrate the differences between buffered and unbuffered channels.

Buffered vs. Unbuffered Channels as Illustrations

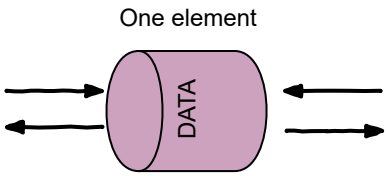
Illustrations of buffered and unbuffered channels to explain the differences between the two.

WE'LL COVER THE FOLLOWING

- Unbuffered Channels
- Buffered Channels

Unbuffered Channels

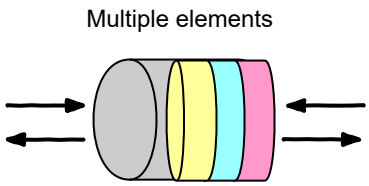
We've been using unbuffered channels in all our previous illustrations. What makes them unique is that only one piece of data fits through the channel at a time.



Unbuffered Channel

Buffered Channels

Buffered channels work just like unbuffered channels, but with one catch—we can send multiple pieces of data to the channel without waiting for another Go routine to read from it.



Buffered Channel

Buffered channels can be useful in concurrent programs as we could run into

a situation where one Go routine can perform multiple tasks in the time it takes another routine to perform a single task. In order to make sure the

former routine doesn’t have to wait until the latter finishes, we can use a buffered channel.

Range and close

This lesson explains the use of range and how it can be used in closing a channel in Go

WE'LL COVER THE FOLLOWING

- Closing a channel
- Example

Closing a channel

A sender can close a channel to indicate that no more values will be sent. Receivers can test whether a channel has been closed by assigning a second parameter to the receive expression: after

```
v, ok := <-ch
```

`ok` is false if there are no more values to receive and the channel is closed.

The loop `for i := range ch` receives values from the channel repeatedly until it is closed.

> **Note:** Only the sender should close a channel, never the receiver. Sending on a closed channel will cause a panic.

> **Another note:** Channels aren't like files; you don't usually need to close them. Closing is only necessary when the receiver must be told there are no more values coming, such as to terminate a range loop.

Example

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
    "fmt"
)

func fibonacci(n int, c chan int) {
    x, y := 0, 1
    for i := 0; i < n; i++ {
        c <- x
        x, y = y, x+y
    }
    close(c)
}

func main() {
    c := make(chan int, 10)
    go fibonacci(cap(c), c)
    for i := range c {
        fmt.Println(i)
    }
}
```

As you can see in above example, the code only outputs **10** values since the value of `c` passed in line **18** is **10**. And you know that the **forloop** will only receive values from channel till it closes, which is after the value of `c` is reached in this case **10**.

In the next lesson we will discuss the concept of *select* in Go.

Select

This lesson explains in detail the use of the select statement in Go, it's default case and concept of timeout

WE'LL COVER THE FOLLOWING

- Select Statement
- Example
- Default case
- Timeout

Select Statement

The `select` statement lets a goroutine wait on multiple communication operations.

A `select` blocks until one of its cases can run, then it executes that case. It chooses one at random if multiple are ready.

Example

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"

func fibonacci(c, quit chan int) {
    x, y := 0, 1
    for {
        select { //waiting on a value for c
        case c <- x:
            x, y = y, x+y
        case <-quit:
            fmt.Println("quit")
            return
```

```
        }
    }
}

func main() {
    c := make(chan int)
    quit := make(chan int)
    go func() {
        for i := 0; i < 10; i++ {
            fmt.Println(<-c)
        }
        quit <- 0
    }()
    fibonacci(c, quit)
}
```

Default case

The default case in a `select` is run if no other case is ready.

Use a `default` case to try a send or receive without blocking:

```
select {
case i := <-c:
    // use i
default:
    // receiving from c would block
}
```

Here's a complete example demonstrating the concept:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
    "fmt"
    "time"
)

func main() {
    tick := time.Tick(100 * time.Millisecond)
    boom := time.After(500 * time.Millisecond)
    for {
        select {
        case <-tick:
```

```
            fmt.Println("tick.")
        case <-boom:
            fmt.Println("BOOM!")

            return
        default:
            fmt.Println("    .")
            time.Sleep(50 * time.Millisecond)
        }
    }
}
```

From the above example, you can see that first, the `default` case executes because none of the other *two* cases is ready. The moment a case is ready, such as `tick` case in the example above, the select command blocks till the case is run and `tick` is printed.

Timeout

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "fmt"
        "log"
        "net/http"
        "time"
)

func main() {
        response := make(chan *http.Response, 1)
        errors := make(chan *error)

        go func() {
                resp, err := http.Get("http://matt.aimonetti.net/")
                if err != nil {
                        errors <- &err
                }
                response <- resp
        }()
        for {
                select {
                case r := <-response:
                        fmt.Printf("%s", r.Body)
                        return
                case err := <-errors:
                        log.Fatal(*err)
```

```
                case <-time.After(200 * time.Millisecond):
                        fmt.Printf("Timed out!")
                        return

                }
        }
}
```

Note that in above example, you won't get a response due to sandboxing.

We are using the `time.After` call as a timeout measure to exit if the request didn't give a response within **200ms**.

Now that you are familiar with *goroutines* and their features. It's time to attempt a few exercise questions present in the next lesson.

Quiz on Concurrency

A quick quiz to test understanding of this chapter!

1

Goroutines run in the same address space?

2

The select statement lets a goroutine wait on multiple communication operations?

3

What does the code below denote?

```
ch <- v
```

4

A channel can be tested if it’s closed. However, only the receiver can close a channel?

5

What does the code below denote?

```
ch := make(chan int, 100)
```

Check Answers

Solved Exercise: Step by Step Guide

This section has a few questions for you try out and test your understanding of concurrency

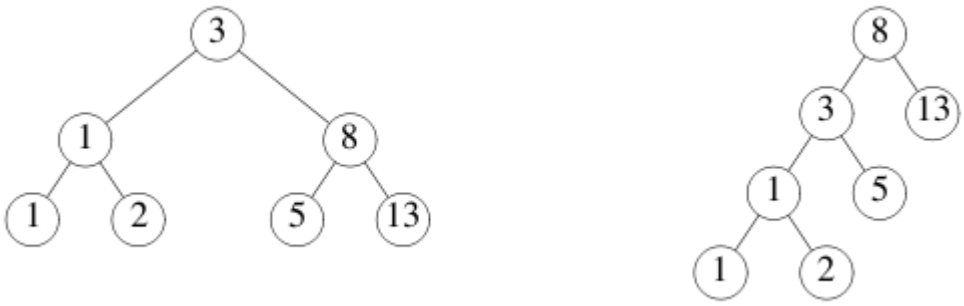
WE'LL COVER THE FOLLOWING

- Assignment
  - Solution

Assignment

Online Assignment

There can be many different binary trees with the same sequence of values stored at the leaves. For example, here are two binary trees storing the sequence 1, 1, 2, 3, 5, 8, 13.



A function to check whether two binary trees store the same sequence is quite complex in most languages. We'll use Go's concurrency and channels to write a simple solution.

This example uses the `tree` package, which defines the type:

```
type Tree struct {
    Left  *Tree
    Value int
    Right *Tree
}
```

1. Implement the Walk function.
2. Test the Walk function.

The function `tree.New(k)` constructs a randomly-structured binary tree holding the values `k`, `2k`, `3k`, ..., `10k`.

Create a new channel `ch` and kick off the walker:

```
go Walk(tree.New(1), ch)
```

Then read and print 10 values from the channel. It should be the numbers `1`, `2`, `3`, ..., `10`.

3. Implement the Same function using `Walk` to determine whether `t1` and `t2` store the same values.

4. Test the `Same` function.

`Same(tree.New(1), tree.New(1))` should return `true`, and `Same(tree.New(1), tree.New(2))` should return `false`.

Solution

If you print `tree.New(1)` you will see the following tree:

```
((((1 (2)) 3 (4)) 5 ((6) 7 ((8) 9))) 10)
```

To implement the `Walk` function, we need two things:

- walk each side of the tree and print the values

- close the channel so the `range` call isn't stuck.

We need to set a recursive call and for that, we are defining a non-exported `recWalk` function, the function walks the left side first, then pushes the value to the channel and then walks the right side. This allows our range to get the values in the right order. Once all branches have been walked, we can close the channel to indicate to the range that the walking is over.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
  "git://github.com/golang/tour"
        "fmt"
)

// Walk walks the tree t sending all values
// from the tree to the channel ch.
func Walk(t *tree.Tree, ch chan int) {
        recWalk(t, ch)
        // closing the channel so range can finish
        close(ch)
}

// recWalk walks recursively through the tree and push values to the channel
// at each recursion
func recWalk(t *tree.Tree, ch chan int) {
        if t != nil {
                // send the left part of the tree to be iterated over first
                recWalk(t.Left, ch)
                // push the value to the channel
                ch <- t.Value
                // send the right part of the tree to be iterated over last
                recWalk(t.Right, ch)
        }
}

// Same determines whether the trees
// t1 and t2 contain the same values.
func Same(t1, t2 *tree.Tree) bool {
        ch1 := make(chan int)
        ch2 := make(chan int)
        go Walk(t1, ch1)
        go Walk(t2, ch2)

        for {
                x1, ok1 := <-ch1
                x2, ok2 := <-ch2
                switch {
                case ok1 != ok2:
                        // not the same size
                        return false
                case !ok1:
                        // both channels are empty
                        return true
                case x1 != x2:
                        // elements are different
                        return false
                default:
                        // keep iterating
                }
        }
}

func main() {
        ch := make(chan int)
```

```go
	go Walk(tree.New(1), ch)
	for v := range ch {
		fmt.Println(v)

	}
	fmt.Println(Same(tree.New(1), tree.New(1)))
	fmt.Println(Same(tree.New(1), tree.New(2)))
}

// Walk walks the tree t sending all values
// from the tree to the channel ch.
func Walk(t *tree.Tree, ch chan int) {
	recWalk(t, ch)
	// closing the channel so range can finish
	close(ch)
}

// recWalk walks recursively through the tree and push values to the channel
// at each recursion
func recWalk(t *tree.Tree, ch chan int) {
	if t != nil {
		// send the left part of the tree to be iterated over first
		recWalk(t.Left, ch)
		// push the value to the channel
		ch <- t.Value
		// send the right part of the tree to be iterated over last
		recWalk(t.Right, ch)
	}
}

// Same determines whether the trees
// t1 and t2 contain the same values.
func Same(t1, t2 *tree.Tree) bool {
	ch1 := make(chan int)
	ch2 := make(chan int)
	go Walk(t1, ch1)
	go Walk(t2, ch2)

	for {
		x1, ok1 := <-ch1
		x2, ok2 := <-ch2
		switch {
		case ok1 != ok2:
			// not the same size
			return false
		case !ok1:
			// both channels are empty
			return true
		case x1 != x2:
			// elements are different
			return false
		default:
			// keep iterating
		}
	}
}

func main() {
	ch := make(chan int)
	go Walk(tree.New(1), ch)
	for v := range ch {
		fmt.Println(v)
	}
```

```
        fmt.Println(Same(tree.New(1), tree.New(1)))
        fmt.Println(Same(tree.New(1), tree.New(2)))
}
```

The comparison of the two trees is trivial once we know how to extract the values of each tree. We just need to loop through the first tree (via the channel), read the value, get the value from the second channel (walking the second tree) and compare the two values.

OSX

Installation guidelines for OSX.

WE'LL COVER THE FOLLOWING

- Homebrew
- Setup your paths
- Install mercurial and bazaar

Homebrew

The easiest way to install Go for development on OS X is to use homebrew.

Using `Terminal.app` install homebrew:

```
$ ruby -e "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/master/install)"
```

Once homebrew is installed, install Go to be able to crosscompile:

```
$ brew install go --cross-compile-common
```

In just a few minutes, go should be all installed and you should be almost ready to code. However we need to do two small things before we start:

Setup your paths

By convention, all your Go code and the code you will import, will live inside a workspace. This convention might seem rigid at first, but it quickly becomes clear that such a convention (like most Go conventions) makes our life much easier.

Before starting, we need to tell Go, where we want our workspace to be, in other words, where our code will live. Let's create a folder named "go" in our

home directory and set our environment to use this location.

```
$ mkdir $HOME/go
$ export GOPATH=$HOME/go
```

Note that if we open a new tab or restart our machine, Go won't know where to find our workspace. For that, you need to set the export in your profile:

```
$ open $HOME/.bash_profile
```

Add a new entry to set `GOPATH` and add the workspace's bin folder to your system path:

```
export GOPATH=$HOME/go
export PATH=$PATH:$GOPATH/bin
```

This will allow your Go workspace to always be set and will allow you to call the binaries you compiled.

Official resource

Install mercurial and bazaar

Optionally but highly recommended, you should install mercurial and bazaar so you can retrieve 3rd party libraries hosted using these version control systems.

```
$ brew install hg bzr
```

Windows

Installation guidelines for Windows.

WE'LL COVER THE FOLLOWING

- Installer
- Official Resource

Installer

Install the latest version by downloading the latest installer.

Official Resource

The official resource for Windows can then be used to install Go.

Linux

Installation guidelines for Linux.

WE'LL COVER THE FOLLOWING

- Official Package

Official Package

Install from one of the official linux packages.

Setup your path, as explained in Section: Setup path

Extras

Additional useful tools in Go.

WE'LL COVER THE FOLLOWING

- Useful Tools
- Official Resource

Useful Tools

Installing `Godoc`, `vet` and `Golint`, three very useful Go tools from the Go team, is highly recommended:

```
$ go get golang.org/x/tools/cmd/godoc
$ go get golang.org/x/tools/cmd/vet
$ go get github.com/golang/lint/golint
```

Official Resource

The official resource for these can be useful.

Coding Challenges

A few samples of exercises you can attempt in order to further polish your skills

WE'LL COVER THE FOLLOWING

- Exercises

Exercises

We have four exercises to choose from, each exercise focuses on different challenges. Try to tackle the exercise of your choice by pairing with at least one person, ideally, try to have four people by group.

Fork the original repo (with the instructions), work on it and send a pull request when you are done.

- Avatar me
  - Hashing
  - image manipulation
- Remote commands
  - Concurrency (channels, go routines)
  - Network interface
  - depending on the commands you implement
- Copernic 2000
  - concurrency
  - consumption of web resources (http/json/XML)
  - sorting
  - data sets
  - data storage

- data storage

- Godoc API
  - Building a web API
  - Testing a web API
  - JSON encoding/decoding
  - Exploring godoc
  - Extending existing packages or
  - Shelling out

Get Your Feet Wet

This lesson gives some additional resources for your learning

WE'LL COVER THE FOLLOWING

- Resource

Resource

One of the best way to learn technical skills is to actually dive in as soon as we have acquired the basics.

This clirescue code available on GitHub was written by someone who just started learning Go. Beginners often make the same mistakes so assume that this is your code and you are now tasked to refactor it. The end goal is to create a CLI to interface with the Pivotal Tracker API.

Fork the project as explained in the readme

Find a one or more people and work with them to see how you would address this refactoring. Time to rescue this project!

BAYWATCH®

Tips for Importing Packages

Useful tips and tricks for programming in Go.

WE'LL COVER THE FOLLOWING

- 140 char tips
- Alternate Ways to Import Packages
- goimports
- Organization
- Custom Constructors
- Breaking down code in packages

This section will grow over time but the main goal is to share some tricks experienced developers discovered over time. Hopefully this tips will get new users more productive faster.

140 char tips

- leave your object oriented brain at home. Embrace the interface. @mikegehard
- Learn to do things the Go way, don't try to force your language idioms into Go. @DrNic
- It's better to over do it with using interfaces than use too few of them. @evanphx
- Embrace the language: simplicity, concurrency, and composition. @francesc
- read all the awesome docs that they have on golang.org. @vbatts
- always use `gofmt`. @darkhelmetlive
- read a lot of source code. @DrNic
- Learn and become familiar with tools and utilities, or create your own! They are as vital to your success as knowing the language. @coreymrak

They are as vital to your success as knowing the language. @coreyprak

Alternate Ways to Import Packages

There are a few other ways of importing packages. We'll use the `fmt` package in the following examples:

- `import format "fmt"` - Creates an alias of `fmt`. Preceed all `fmt` package content with `format.` instead of `fmt.`.
- `import . "fmt"` - Allows content of the package to be accessed directly, without the need for it to be preceded with `fmt`.
- `import _ "fmt"` - Suppresses compiler warnings related to `fmt` if it is not being used, and executes initialization functions if there are any. The remainder of `fmt` is inaccessible.

See this blog post for more detailed information.

goimports

Goimports is a tool that updates your Go import lines, adding missing ones and removing unreferenced ones.

It acts the same as gofmt (drop-in replacement) but in addition to code formatting, also fixes imports.

Organization

Go is a pretty easy programming language to learn but the hardest thing for developers at first is how to organize their code. Rails became popular for many reasons and scaffolding was one of them. It gave new developers clear directions and places to put their code and idioms to follow.

To some extent, Go does the same thing by providing developers with great tools like `go fmt` and by having a strict compiler that won't compile unused variables or unused import statements.

Custom Constructors

A question I often hear is when should I use custom constructors like `NewJob`. My answer is that in most cases you don't need to. However, whenever you

need to set your value at initialization time and you have some sort of default values, it's a good candidate for a constructor. In the above example, adding a constructor makes a lot of sense so we can set a default logger.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "log"
        "os"
)

type Job struct {
        Command string
        *log.Logger
}

func NewJob(command string) *Job {
        return &Job{command, log.New(os.Stdout, "Job: ", log.Ldate)}
}

func main() {
        NewJob("demo").Print("starting now...")
}
```

Breaking down code in packages

See this blog post on refactoring Go code, the first part talks about package organization.

Tips on Maps & Errors

Useful tips and tricks for programming in Go.

WE'LL COVER THE FOLLOWING

- Sets
- Dependency package management
- Using errors

Sets

You might want to find a way to extract unique value from a collection. In other languages, you often have a set data structure not allowing duplicates. Go doesn't have that built in, however it's not too hard to implement (due to a lack of generics, you do need to do that for most types, which can be cumbersome).

```
// UniqStr returns a copy if the passed slice with only unique string results.
func UniqStr(col []string) []string {
        m := map[string]struct{}{}
        for _, v := range col {
                if _, ok := m[v]; !ok {
                        m[v] = struct{}{}
                }
        }
        list := make([]string, len(m))

        i := 0
        for v := range m {
                list[i] = v
                i++
        }
        return list
}
```

I used a few interesting tricks that are interesting to know. First, the map of empty structs:

```
m := map[string]struct{}{}
```

We create a map with the keys being the values we want to be unique, the associated value doesn't really matter much so it could be anything. For instance:

```
m := map[string]bool{}
```

However I chose an empty structure because it will be as fast as a boolean but doesn't allocate as much memory.

The second trick can been seen a bit further:

```
if _, ok := m[v]; !ok {
  m[v] = struct{}{}
}
```

What we are doing here, is simply check if there is a value associated with the key `v` in the map `m`, we don't care about the value itself, but if we know that we don't have a value, then we add one.

Once we have a map with unique keys, we can extract them into a new slice of strings and return the result.

Here is the test for this function, as you can see, I used a table test, which is the idiomatic Go way to run unit tests:

Environment Variables

Key:	Value:
GOPATH	/go

```
func TestUniqStr(t *testing.T) {

        data := []struct{ in, out []string }{
                {[]string{}, []string{}},
                {[]string{"", "", ""}, []string{""}},
                {[]string{"a", "a"}, []string{"a"}},
                {[]string{"a", "b", "a"}, []string{"a", "b"}},
                {[]string{"a", "b", "a", "b"}, []string{"a", "b"}},
                {[]string{"a", "b", "b", "a", "b"}, []string{"a", "b"}},
                {[]string{"a", "a", "b", "b", "a", "b"}, []string{"a", "b"}},
                {[]string{"a", "b", "c", "a", "b", "c"}, []string{"a", "b", "c"}},
        }
```

```
        for _, exp := range data {
                res := UniqStr(exp.in)
                if !reflect.DeepEqual(res, exp.out) {
                        t.Fatalf("%q didn't match %q\n", res, exp.out) //produce an error if
                }
        }

}
```

Given below is the implementation of the `UniqueStr()` method and the test code that logs a runtime error if the output from the method does not match the test cases:

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import (
        "log"
        "reflect"
)

// UniqStr returns a copy if the passed slice with only unique string results.
func UniqStr(col []string) []string {
        m := map[string]struct{}{}
        for _, v := range col {
                _, ok := m[v]
                if !ok {
                        m[v] = struct{}{}
                }
        }
        list := make([]string, len(m))

        i := 0
        for v := range m {
                list[i] = v
                i++
        }
        return list
}

func main() {
        data := []struct{ in, out []string }{
                {[]string{}, []string{}},
                {[]string{"", "", ""}, []string{""}},
                {[]string{"a", "a"}, []string{"a"}},
                {[]string{"a", "b", "a"}, []string{"a", "b"}},
                {[]string{"a", "b", "a", "b"}, []string{"a", "b"}},
                {[]string{"a", "b", "b", "a", "b"}, []string{"a", "b"}},
                {[]string{"a", "a", "b", "b", "a", "b"}, []string{"a", "b"}},
                {[]string{"a", "b", "c", "a", "b", "c"}, []string{"a", "b", "c"}},
        }
```

```
        for _, exp := range data {
                res := UniqStr(exp.in)
                if !reflect.DeepEqual(res, exp.out) {
                        log.Fatalf("%q didn't match %q\n", res, exp.out) //log a runtime erro
                }
        }
}
```

Dependency package management

Unfortunately, Go doesn't ship with its own dependency package management system. Probably due to its roots in the C culture, packages aren't versioned and explicit version dependencies aren't addressed.

The challenge is that if you have multiple developers on your project, you want all of them to be on the same version of your dependencies. Your dependencies might also have their own dependencies and you want to make sure everything is in a good state. It gets even tricker when you have multiple projects using different versions of the same dependency. This is typically the case in a CI environment.

The Go community came up with a lot of different solutions for these problems. But for me, none are really great so at Splice we went for the simplest working solution we found: gpm

Gpm is a simple bash script, we end up modifying it a little so we could drop the script in each repo. The bash script uses a custom file called `Godeps` which lists the packages to install.

When switching to a different project, we run the project `gpm` script to pull down or set the right revision of each package.

In our CI environment, we set `GOPATH` to a project specific folder before running the test suite so packages aren't shared between projects.

Using errors

Errors are very important pattern in Go and at first, new developers are surprised by the amount of functions returning a value and an error.

Go doesn't have a concept of an exception like you might have seen in other

Go doesn't have a concept of an exception like you might have seen in other programming languages. Go does have something called `panic` but as its

name suggests they are really exceptional and shouldn't be rescued (that said, they can be).

The error handling in Go seems cumbersome and repetitive at first, but quickly becomes part of the way we think. Instead of creating exceptions that bubble up and might or might not be handled or passed higher, errors are part of the response and designed to be handled by the caller. Whenever a function might generate an error, its response should contain an error param.

Andrew Gerrand from the Go team wrote a great blog post on errors I strongly recommend you read it.

Effective Go section on errors

Tips on Compiler Optimization

Useful tips and tricks for programming in Go.

WE'LL COVER THE FOLLOWING

- Compiler Optimizations
- Expvar
- Setting the Build ID using git's SHA
- How to see what packages my app imports

Compiler Optimizations

You can pass specific compiler flags to see what optimizations are being applied as well as how some aspects of memory management. This is an advanced feature, mainly for people who want to understand some of the compiler optimizations in place.

Let's take the following code example from an earlier chapter:

```
package main

import "fmt"

type User struct {
        Id             int
        Name, Location string
}

func (u *User) Greetings() string {
        return fmt.Sprintf("Hi %s from %s",
                u.Name, u.Location)
}

func NewUser(id int, name, location string) *User {
        id++
        return &User{id, name, location}
}

func main() {
        u := NewUser(42, "Matt", "LA")
```

```
        fmt.Println(u.Greetings())
}
```

Build your file (here called `t.go`) passing some `gcflags`:

```
$ go build -gcflags=-m t.go
# command-line-arguments
./t.go:15: can inline NewUser
./t.go:21: inlining call to NewUser
./t.go:10: leaking param: u
./t.go:10: leaking param: u
./t.go:12: (*User).Greetings ... argument does not escape
./t.go:15: leaking param: name
./t.go:15: leaking param: location
./t.go:17: &User literal escapes to heap
./t.go:15: leaking param: name
./t.go:15: leaking param: location
./t.go:21: &User literal escapes to heap
./t.go:22: main ... argument does not escape
```

The compiler notices that it can inline the `NewUser` function defined on line 15 and inline it on line 21. Dave Cheney has a great post about why Go's inlining is helping your programs run faster.

Basically, the compiler moves the body of the `NewUser` function (L15) to where it's being called (L21) and therefore avoiding the overhead of a function call but increasing the binary size.

The compiler creates the equivalent of:

```
func main() {
        id := 42 + 1
        u := &User{id, "Matt", "LA"}
        fmt.Println(u.Greetings())
}
```

On a few lines, you see the potentially alarming `leaking param` message. It doesn't mean that there is a memory leak but that the param is kept alive even after returning. The "leaked params" are:

- On the `Greetings`'s method: `u` (receiver)
- On the `NewUser`'s functon: `name`, `location`

The reason why `u` "leaks" in the `Greetings` method is because it's being used

The reason why `u` "leaks" in the `Greetings` method is because it's being used in the `fmt.Sprintf` function call as an argument. `name` and `location` are also

"leaked" because they are used in the `User`'s literal value. Note that `id` doesn't leak because it's a value, only references and pointers can leak.

X `argument does not escape` means that the argument doesn't "escape" the function, meaning that it's not used outside of the function so it's safe to store it on the stack.

On the other hand, you can see that `&User literal escapes to heap`. What it means is that the address of a literal value is used outside of the function and therefore can't be stored on the stack. The value *could* be stored on the stack, except a pointer to the value escapes the function, so the value has to be moved to the heap to prevent the pointer referring to incorrect memory once the function returns. This is always the case when calling a method on a value and the method uses one or more fields.

Expvar

TODO package

Setting the Build ID using git's SHA

It's often very useful to burn a build id in your binaries. I personally like to use the SHA1 of the git commit I'm committing. You can get the short version of the sha1 of your latest commit by running the following `git` command from your repo:

```
git rev-parse --short HEAD
```

The next step is to set an exported variable that you will set at compilation time using the `-ldflags` flag.

Environment Variables

Key:	Value:
GOPATH	/go

```
package main

import "fmt"
```

```
// compile passing -ldflags "-X main.Build <build sha1>"
var Build string


func main() {
        fmt.Printf("Using build: %s\n", Build)
}
```

Save the above code in a file called `example.go`. If you run the above code, `Build` won't be set, for that you need to set it using `go build` and the `-ldflags`.

```
$ go build -ldflags "-X main.Build a1064bc" example.go
```

Now run it to make sure:

```
$ ./example
Using build: a1064bc
```

Now, hook that into your deployment compilation process, I personally like Rake to do that, and this way, every time I compile, I think of Jim Weirich.

How to see what packages my app imports

\label{sec:list_imported_go_packages}

It's often practical to see what packages your app is importing. Unfortunately there isn't a simple way to do that, however it is doable via the `go list` tool and using templates.

Go to your app and run the following.

```
$ go list -f '{{join .Deps "\n"}}' |
  xargs go list -f '{{if not .Standard}}{{.ImportPath}}{{end}}'
```

Here is an example with the clirescue refactoring example:

```
$ cd $GOPATH/src/github.com/GoBootcamp/clirescue
$ go list -f '{{join .Deps "\n"}}' |
  xargs go list -f '{{if not .Standard}}{{.ImportPath}}{{end}}'
github.com/GoBootcamp/clirescue/cmdutil
github.com/GoBootcamp/clirescue/trackerapi
github.com/GoBootcamp/clirescue/user
github.com/codegangsta/cli
```

If you want the list to also contain standard packages, edit the template and use:

```
$ go list -f '{{join .Deps "\n"}}' |  xargs go list -f '{{.ImportPath}}'
```

Tips on Constants in Go

Useful tips and tricks for programming in Go.

WE'LL COVER THE FOLLOWING

- Iota: Elegant Constants
  - Auto Increment
  - Custom Types
  - Skipping Values
  - Expressions
- Web resources

Iota: Elegant Constants

Some concepts have names, and sometimes we care about those names, even (or especially) in our code.

```
const (
    CCVisa            = "Visa"
    CCMasterCard      = "MasterCard"
    CCAmericanExpress = "American Express"
)
```

At other times, we only care to distinguish one thing from the other. There are times when there's no inherently meaningful value for a thing. For example, if we're storing products in a database table we probably don't want to store their category as a string. We don't care how the categories are named, and besides, marketing changes the names all the time.

We care only that they're distinct from each other.

```
const (
    CategoryBooks    = 0
    CategoryHealth   = 1
```

```
    CategoryClothing = 2
)
```

Instead of 0, 1, and 2 we could have chosen 17, 43, and 61. The values are arbitrary.

Constants are important but they can be hard to reason about and difficult to maintain. In some languages like Ruby developers often just avoid them. In Go, constants have many interesting subtleties that, when used well, can make the code both elegant and maintainable.

Auto Increment

A handy idiom for this in golang is to use the iota identifier, which simplifies constant definitions that use incrementing numbers, giving the categories exactly the same values as above.

```
const (
    CategoryBooks = iota // 0
    CategoryHealth       // 1
    CategoryClothing     // 2
)
```

Custom Types

Auto-incrementing constants are often combined with a custom type, allowing you to lean on the compiler.

```
type Stereotype int

const (
    TypicalNoob Stereotype = iota // 0
    TypicalHipster                // 1
    TypicalUnixWizard             // 2
    TypicalStartupFounder         // 3
)
```

If a function is defined to take an int as an argument rather than a Stereotype, it will blow up at compile-time if you pass it a Stereotype:

```
func CountAllTheThings(i int) string {
                return fmt.Sprintf("there are %d things", i)
}

func main() {
    n := TypicalHipster
    fmt.Println(CountAllTheThings(n)) // cannot use TypicalHipster (type Stereotype) as type
```

```
    fmt.Println(CountAllTheThings(h)) // cannot use TypicalHipster (type Stereotype) as type
}
```

The inverse is also true. Given a function that takes a Stereotype as an argument, you **can't** pass it an int:

```
func SoSayethThe(character Stereotype) string {
    var s string
    switch character {
    case TypicalNoob:
        s = "I'm a confused ninja rockstar."
    case TypicalHipster:
        s = "Everything was better we programmed uphill and barefoot in the snow on the SUTX
    case TypicalUnixWizard:
        s = "sudo grep awk sed %!#?!!1!"
    case TypicalStartupFounder:
        s = "exploit compelling convergence to syndicate geo-targeted solutions"
    }
    return s
}

func main() {
    i := 2
    fmt.Println(SoSayethThe(i)) // cannot use i (type int) as type Stereotype in argument to
}
```

There's a dramatic twist, however. You could pass a number constant, and it would work:

```
func main() {
    fmt.Println(SoSayethThe(0))
}

// output:
// I'm a confused ninja rockstar.
```

This is because constants in Go are loosely typed until they are used in a strict context.

Skipping Values

Imagine that you're dealing with consumer audio output. The audio might not have any output whatsoever, or it could be mono, stereo, or surround.

There's some underlying logic to defining no output as 0, mono as 1, and stereo as 2, where the value is the number of channels provided.

So what value do you give Dolby 5.1 surround?

On the one hand, it's 6 channel output, but on the other hand, only 5 of those channels are full bandwidth channels (hence the 5.1 designation – with the .1 referring to the low-frequency effects channel).

Either way, we don't want to simply increment to 3.

We can use underscores to skip past the unwanted values.

```
type AudioOutput int

const (
    OutMute AudioOutput = iota // 0
    OutMono                    // 1
    OutStereo                  // 2
    _
    _
    OutSurround                // 5
)
```

Expressions

The iota can do more than just increment. Or rather, iota always increments, but it can be used in expressions, storing the resulting value in the constant.

Here we're creating constants to be used as a bitmask.

```
type Allergen int

const (
    IgEggs Allergen = 1 << iota // 1 << 0 which is 00000001
    IgChocolate                 // 1 << 1 which is 00000010
    IgNuts                      // 1 << 2 which is 00000100
    IgStrawberries              // 1 << 3 which is 00001000
    IgShellfish                 // 1 << 4 which is 00010000
)
```

This works because when you have only an identifier on a line in a const group, it will take the previous expression and reapply it, with the incremented iota. In the language of the spec, this is called *implicit repetition*

of the last non-empty expression list.

If you're allergic to eggs, chocolate, and shellfish, and flip those bits to the "on" position (mapping the bits right to left), then you get a bit value of 00010011, which corresponds to 19 in decimal.

```
fmt.Println(IgEggs | IgChocolate | IgShellfish)

// output:
// 19
```

There's a great example in Effective Go for defining orders of magnitude:

```
type ByteSize float64

const (
    _           = iota              // ignore first value by assigning to blank identifier
    KB ByteSize = 1 << (10 * iota)  // 1 << (10*1)
    MB                              // 1 << (10*2)
    GB                              // 1 << (10*3)
    TB                              // 1 << (10*4)
    PB                              // 1 << (10*5)
    EB                              // 1 << (10*6)
    ZB                              // 1 << (10*7)
    YB                              // 1 << (10*8)
)
```

Today I learned that after zettabyte we get yottabyte. #TIL

But Wait, There's More

What happens if you define two constants on the same line? What is the value of Banana? 2 or 3? And what about Durian?

```
const (
    Apple, Banana = iota + 1, iota + 2
    Cherimoya, Durian
    Elderberry, Fig
)
```

The iota increments on the next line, rather than as soon as it gets referenced.

```
// Apple: 1
// Banana: 2
// Cherimoya: 2
// Durian: 3
// Elderberry: 3
```

```
// Fig: 4
```

Which is messed up, because now you have constants with the same value.

So, Yeah

There's a lot more to be said about constants in Go, and you should probably read Rob Pike's blog post on the subject over on the golang blog.

This was first published by Katrina Owen on the Splice blog.

Web resources

- Dave Cheney maintains a list of resources for new Go developers.